



UNIVERSITAT
POLITÈCNICA
DE VALÈNCIA

AVIÓNICA

Software



Authors:

Román Galiana Romero

EJERCICIO 1: Selección de microcontrolador y configuración básica

En el departamento de aviónica, trabajamos principalmente con microcontroladores del ecosistema STM32.

- a. **Selecciona un microcontrolador STM32 para la aviónica de vuelo y justifica tu elección (memoria, potencia de cálculo, consumo, periféricos, etc.).**

Los microcontroladores STM32 son una familia de micros basada en la arquitectura de 32 bits ARM Cortex-M, que vienen perfectos para la placa de vuelo porque procesan matemáticas por hardware muy rápidamente gracias a su FPU (Floating Point Unit) que son un bloque de transistores dedicado exclusivamente a calcular con decimales en precisión simple de 32 bits que le permite multiplicar dos números con decimales en único ciclo de reloj (a 168 MHz, unos 6 nanosegundos). También son de bajo consumo y están diseñados para reaccionar a interrupciones de forma prácticamente instantánea sin quedarse colgado, gracias a su gestor de interrupciones por hardware llamado NVIC (Nested Vectored Interrupt Controller) haciendo que ante un evento crítico, pare lo que esté haciendo en ese momento y salte a atender la emergencia en 12 ciclos de reloj (del orden de 70 nanosegundos a 168MHz), volviendo una vez terminado a la instrucción donde se había quedado de manera robusta y sin retardos que afecten al control del vuelo.

Sin embargo, hay que tener mucho cuidado al elegir el modelo correcto dentro de esta familia para operar un sistema de control de vuelo tan exigente. El microcontrolador debe tener una gran RAM para evitar un desbordamiento de pila (stack overflow) para ser capaz de soportar un Sistema Operativo en Tiempo Real (FreeRTOS), en el cual cada tarea (como por ejemplo leer sensores, calcular navegación, escribir en la SD o transmitir telemetría) necesita su propia reserva de memoria RAM (stack)*. También debe ser capaz de resolver el filtro de Kalman para la fusión de las medidas de los sensores y estimar el estado del cohete (altitud, velocidad vertical, orientación, etc), el cual pide una gran capacidad matemática (la FPU que lleva el Cortex-M viene de lujo para esto) y también debe ser capaz de gestionar la telemetría sin latencia (tiempo de retraso desde que da una orden hasta que se obtiene una respuesta o el dato llegue a su destino) y el datalogger (el sistema encargado de grabar y almacenar todas las medidas de los sensores y variables del sistema en la tarjeta MicroSD para el análisis postvuelo) mediante acceso directo a memoria (DMA*). Respecto a esto último, enviar datos por radio y transmitir los registros de datos almacenados en la RAM a la MicroSD a través de los buses SPI y UART consume muchísimo tiempo si lo hace la CPU a mano. Con los canales DMA (Direct Memory Access), el hardware transfiere estos paquetes en segundo plano directamente desde la RAM, dejando al procesador libre al 100% para guiar el cohete y vigilar el apogeo.

**El stack (pila) es un apartado de memoria donde el procesador guarda la gestión interna de cada tarea como las variables locales de las funciones, los argumentos de entrada de las funciones y las direcciones de retorno para volver a donde estaba antes,*

una vez terminado de la una función. Puede ser que una tarea empiece llamar a funciones dentro de funciones, declare variables locales muy grandes o entre una interrupción haciendo que la pila crezca bastante y haciendo que en casos concretos sobrepase el límite de pila asignado a una tarea haciendo que se desborde la pila (stack overflow), y haciendo que se escriba en la memoria asignada a otras tareas o incluso del propio sistema operativo, corrompiendo datos críticos y pudiendo provocar que el procesador entre en fallo crítico permanente (conocido como HardAult) y se quede congelado en pleno vuelo.

** El DMA es un circuito que conecta directamente la memoria RAM con los periféricos (SPI, UART) sin que la CPU tenga que intervenir byte a byte. Para ello la CPU le dice al DMA en que dirección de RAM empieza el paquete de datos (el búfer de la RAM) y cuantos bytes tiene que transferir hacia el bus. Por ello el micro también requiere un gran espacio en la RAM para que el DMA tenga de donde leer y donde escribir.*

Dentro de la gama de alto rendimiento, he evaluado dos microcontroladores principales: el STM32F405 y el STM32F411. Aunque el STM32F411 es más eficiente y por lo tanto tiene un consumo menor, cuenta con bastantes menos puertos periféricos (solo 3 UARTs) y bastante menos memoria RAM. Por este motivo, la elección definitiva es el STM32F405. Este chip cuenta con 1 MB de memoria Flash y 192 KB de RAM, frente a los limitados 128 KB de RAM del F411 (un 50% más).

Como he dicho anteriormente, la necesidad de una gran memoria es crítica, pero ya no solo para poder gestionar el FreeRTOS, sino porque escribir datos a alta velocidad en la MicroSD (para el datalogger) requiere crear grandes búferes en la RAM. El DMA (el cual los dos microcontroladores también cuentan con dos controladores DMA) se encarga de mover la información en segundo plano dejando libre a la CPU para hacer otras tareas, pero necesita espacio en la memoria para almacenar los bloques de escritura de la SD (para el buffer) * y los búferes asíncronos de recepción del GPS (UART). Si sumamos estos búferes de comunicación, las matrices dinámicas del filtro de Kalman Extendido (en el ejercicio 3 explico de que consiste el EFK) y las reservas de pila (stack) que exige cada tarea de FreeRTOS, los 128 KB del F411 se quedan cortos.

** Un buffer es un espacio reservado en la memoria RAM para almacenar los datos temporalmente mientras pasan de un sitio a otro. Si el flujo de datos que recibe la RAM es más rápido de lo que el bus puede enviar los datos que ya están esperando a irse, y el tamaño asignado al buffer se queda corto (el espacio reservado en memoria para el buffer), se produce lo que se llama como un desbordamiento de buffer (buffer overflow), que hace que los datos nuevos sobrescriben la otra memoria de la RAM (igual que ocurría con el stack overflow) pudiendo perderse variables críticas del sistema e incluso haciendo que el microcontrolador se reinicie o se congele en pleno vuelo. Para ello hace falta una gran memoria RAM para poder reservar grandes apartados de la memoria para los búferes y evitar el Buffer Overflow.*

Todo esto que acabo de explicar sobre el stack overflow y el buffer overflow exige una gran capacidad de memoria RAM (memoria volátil) * para evitar que el procesador entre en un fallo crítico permanente (HardFault). Si este bloqueo ocurriese antes del apogeo, el micro no podría ordenar la apertura del paracaídas, provocando que el cohete caiga

a cientos de km/h contra el suelo, destrozando el vehículo por completo y suponiendo un riesgo grave de seguridad.

Por eso, los 192 KB de RAM del STM32F405 dan el margen de seguridad de sobra que necesitamos para que el código corra rápido, sin latencia (pausas) y asegurarnos de que el sistema responderá a tiempo en cada fase del vuelo.

**La memoria volátil es la memoria RAM. Es la memoria que en cuanto le quitas la alimentación al microcontrolador se borra todo (de ahí el nombre volátil), al contrario de la memoria Flash que conserva el código grabado. La memoria Flash, en cambio, es donde el microcontrolador guarda todo el código compilado y almacena todas las constantes del programa. Este modelo cuenta con 1 MB de esta memoria, lo que le da capacidad de sobra para meter todas las librerías necesarias, el núcleo de FreeRTOS y el código principal de vuelo sin preocuparse nunca por quedarse sin espacio de almacenamiento.*

Respecto a la potencia de cálculo, el STM32F405 opera con un núcleo a 168 MHz e integra una Unidad de Coma Flotante (FPU) física de 32 bits como he comentado antes, siendo este un gran requisito para un cohete sonda, ya que permite resolver las multiplicaciones de matrices y las derivadas parciales (el Jacobiano) del Filtro de Kalman de forma rápida y robusta. Si no tendría FPU, el procesador solo sabría operar con números enteros y requeriría resolver esto con código, descomponiendo por ejemplo una simple multiplicación de matrices con decimales en decenas o cientos de operaciones básicas con enteros, tardando del orden de 100 ciclos en lo que el hardware del FPU puede hacer en 1 solo ciclo.

En cuanto a los periféricos, el F405 tiene 3 buses I2C, 3 buses SPI y 6 puertos UART, haciéndolo más que suficiente para cumplir con la redundancia de hardware que exige un sistema de control de vuelo de un cohete.

- Los buses I2C permiten conectar más de un sensor al mismo par de cables que entran a los pines de un mismo bus del microcontrolador pines (la línea de reloj SCL que marca el ritmo en el que se transmiten y leen los datos, y la línea de datos SDA que es donde se envían los bits de lectura) gracias al protocolo de direccionamiento físico del bus I2C*. Esto ahorra un muchísimo espacio en el diseño de la PCB ya que en vez de tener que conectar circuitos independientes para cada sensor, puedes conectar varios sensores a esas dos únicas líneas de entrada al micro.

**El I2C trabaja llamando al sensor con el que se quiere comunicar el micro, transmitiendo la dirección que tiene asignada ese sensor específico por la línea SDA y así asegurándose que el único sensor que tiene esa dirección asignada se conecte a la línea mientras los demás sensores mantienen desconectados su salida de datos.*

- Los puertos UART conectan directamente un chip con otro sin necesidad de cables de reloj compartidos, siendo ideales para recibir datos continuos del GPS o transmitir telemetría por radio a tierra. Solo utilizan dos líneas cruzadas donde lo que un dispositivo emite por su pin TX (Transmitter) entra directo al pin RX

(Receiver) del otro. Al ser una comunicación asíncrona que carece de una señal que marque el ritmo de los bits, tanto el transmisor como receptor se configuran primero a la misma velocidad de transmisión en baudios (bits/s). El emisor envía un bit de inicio para avisar de que entra un dato, suelta los bits de información y termina con un bit de parada para cerrar el paquete, permitiendo intercambiar información en segundo plano de forma simple y sin depender de sincronizaciones externas.

- Los buses SPI, gracias a su altísima velocidad y los pines de selección (Chip Select), permiten escribir miles de variables por segundo en la MicroSD (el datalogger) y leer sensores rápidos como la IMU al instante, sin que se formen atascos de datos que generen latencias en el control de vuelo. Para conseguir esto, el bus SPI utiliza varias líneas: el pin CS (Chip Select) es un cable exclusivo para cada sensor que va del microcontrolador a cada sensor y permite que cuando el micro quiere hablar con un sensor específico, activa su pin CS para decirle que le toca hablar, mientras los demás sensores se quedan en silencio sin interferir. Luego, para mover la información utiliza dos vías independientes de sentido único: MOSI (Master Out, Slave In), por donde el microcontrolador envía órdenes o datos al sensor, y MISO (Master In, Slave Out), por donde el sensor le devuelve las lecturas al micro. Al tener una línea para emitir y otra distinta para recibir a la vez, la comunicación es instantánea y no pierde tiempo esperando turnos.

En cambio, el STM32F411 tiene solo 3 puertos UART y si conectamos el GPS a uno y la radio de telemetría a otro, ya nos quedándonos casi sin puertos. El F405, con sus 6 UARTs permite que el diseño actual pueda crecer en el futuro para añadir un GPS redundante, por ejemplo, una cámara o cualquier otro módulo sin tener que cambiar de placa.

Además, el F411 no tiene ni un solo puerto de bus CAN. El F405 en cambio viene con 2 puertos CAN de fábrica, que es un tipo de conexión fundamental en aeroespacial porque aguanta el ruido electromagnético de maravilla gracias a que envía la señal por dos cables a la vez (de manera diferencial) *. Si más adelante el cohete se hace más grande, pasa a tener varias etapas o necesitamos tirar cables largos por dentro del tubo para controlar las válvulas de propulsión (el mismo bus CAN ya se diseñó para recorrer varios metros), el F405 ya viene preparado para hablar con todos esos módulos sin tener que tirar a la basura el diseño y volver a fabricar la placa desde cero.

**El bus CAN funciona transmitiendo la información mediante dos cables trenzados (CAN High y CAN Low) con voltajes opuestos. Si una interferencia electromagnética o el ruido de los motores entra en el cable por pasar cerca del motor, afecta a las dos líneas por igual ya que van trenzados, por lo que el receptor simplemente resta las dos señales y cancela el ruido por completo, asegurando que las órdenes de por ejemplo las válvulas o los datos entre etapas nunca lleguen corruptos. (si el cohete tiene dos etapas, la primera etapa necesite mandar datos de sus sensores y recibir órdenes del micro que se suele encontrar en la etapa superior)*

Sin embargo, aunque el STM32F411 es más eficiente ya que operando a una frecuencia

máxima de 100 MHz consume tan solo unos 10 a 12 mA, frente a los aproximadamente 40 mA que consumiría el F405 a su máxima frecuencia de 168 MHz, este ahorro de apenas 30 mA no justifica la pérdida de 68 MHz de velocidad de cálculo (vital para ejecutar el Filtro de Kalman sin latencia) ni el recorte drástico en sus periféricos.

Considerando además que la aviónica se alimentará con paquetes de baterías LiPo de alta capacidad (como los 5800 mAh propuestos en ejercicios de hardware), el cohete podría pasar horas de espera en la rampa de lanzamiento sin consumir casi la batería. Por lo tanto, he concluido que, aunque el F405 tenga un consumo un poco mayor, sigue siendo un microcontrolador muy eficiente que garantiza energía de sobra para operar los subsistemas y llegar al apogeo con energía suficiente para lanzar el paracaídas y enviar sus coordenadas a la estación en tierra hasta su aterrizaje.

Por último, el STM32F405 es el cerebro que llevan montado muchos controladores de vuelo de referencia en el mundo, tanto en el sector profesional como en proyectos abiertos*. Con esto sabemos que este micro está más que probado con millones de horas reales de vuelo resolviendo algoritmos de control y orientación (IMU, PID y filtros de Kalman), y no nos va a dar ningún susto raro al mover datos a miles de Hz por los buses.

**Los controladores de vuelo comerciales y los proyectos de código abierto han utilizado este microcontrolador durante años en todo tipo de condiciones de vibración, aceleración y temperatura por lo que significa que los fallos de diseño del propio hardware (erratas) y los límites de sus periféricos ya están totalmente documentados y resueltos por la comunidad, evitando encontrarnos con comportamientos inesperados en pleno lanzamiento.*

- b. **Configura un proyecto básico en STM32CubeMX, que cuente con:**
- **Al menos un bus SPI.**
 - **Al menos un bus I²C.**
 - **La configuración necesaria para poder utilizar un programador ST-LINK.**

He implementado un proyecto base sobre el microcontrolador STM32F405RGT6 (encapsulado LQFP64)* con la configuración requerida para la arquitectura de la aviónica:

** RGT6. La R significa que tiene 64 pines, G significa 1 Megabyte de memoria Flash interna para almacenar el código, la T es el tipo de encapsulado que es de tipo LQFP y el 6 es el rango de temperatura al que trabaja que es el grado industrial estándar de -40 °C a +85 °C. El encapsulado es la carcasa física de plástico o resina que lo protege por fuera y saca sus conexiones eléctricas hacia patillas metálicas para poder soldarlo a la placa PCB. Las siglas LQFP64 definen la forma física que tiene esa carcasa: la L viene de Low-profile (perfil bajo) que indica un grosor de solo 1.4 mm de altura, la Q de Quad significa que tiene patillas por sus cuatro lados, la FP de Flat Package indica que las patillas salen planas hacia los lados doblándose un poco hacia abajo con forma de pata para apoyarse en las pistas de la placa mediante tecnología de montaje superficial o SMD (Surface Mount Device-dispositivo de montaje superficial que significa que está diseñado para soldarse directamente sobre la superficie exterior de la placa PCB sin necesidad de atravesarla con cable), y el 64 confirma el número total de pines distribuidos en sus cuatro caras.*

- **Depuración y flasheo (SWD con ST-LINK):**

En la sección System Core dentro del menú SYS active el modo Serial Wire Debug (SWD), que significa depuración por cable serie. Esta configuración reserva directamente los pines PA13 como SWDIO, que es la línea de entrada y salida de datos de depuración Serial Wire Data Input Output, y PA14 como SWCLK, que corresponde a la señal de reloj Serial Wire Clock. Estos dos pines son la puerta de acceso física que conecta el módulo interno de depuración del procesador con el programador ST-LINK a través de un USB. Con esta conexión podemos flashear o grabar el código binario compilado en la memoria Flash no volátil y depurar el código (hacer debugging) en tiempo real, lo que permite pausar la CPU, rastrear fallos en el código, vigilar el valor de las variables y registros directamente en la memoria para identificar cualquier fallo de funcionamiento antes del vuelo. Activar esta opción de SWD es necesario para evitar que el código autogenerado reasigna esos pines como entradas o salidas digitales de propósito general (GPIO) nada más arrancar la placa, lo que desconecta internamente la interfaz de depuración, haciendo que el programador ST-Link no encuentre el micro, generando problemas para reprogramar el microcontrolador de forma normal.

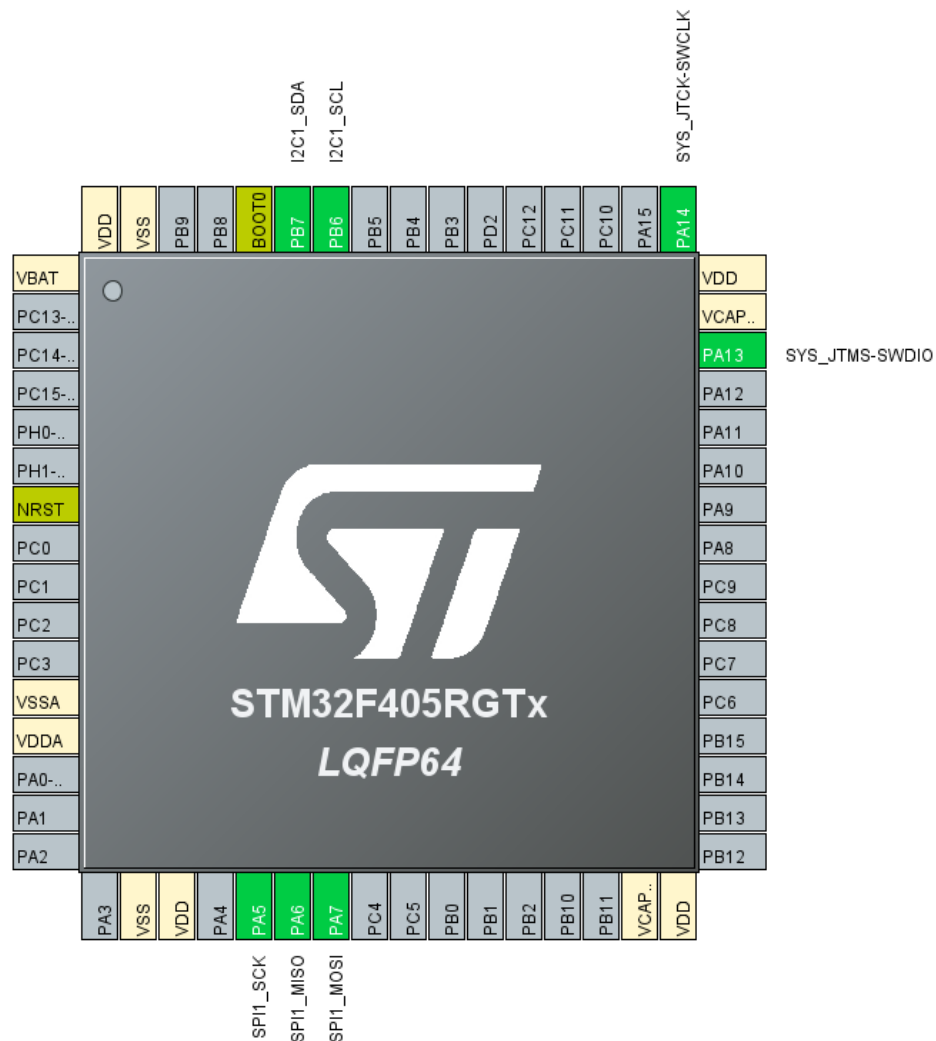
- **Bus I2C:**

En la pestaña Connectivity seleccionamos el periférico I2C1 configurado en modo estándar de I2C, que es el protocolo Inter Integrated Circuit que ya he explicado antes. He asignado el pin PB6 a la señal I2C1 SCL (la línea de reloj), y el pin PB7 a la señal I2C1

SDA (la línea de datos). Este bus opera en modo drenador abierto (open drain) junto con resistencias pull up conectadas a la línea de alimentación de 3.3V. En esta configuración los transistores internos de los dispositivos solo pueden forzar el nivel bajo a cero voltios, mientras que para marcar un nivel alto simplemente desconectan su circuito interno y dejan que una resistencia externa tire del voltaje hacia arriba de manera pasiva. Esto previene cortocircuitos si dos sensores intentan transmitir a la vez ya que ninguno inyecta corriente positiva directamente sobre el cable, que permite que varios dispositivos compartan las mismas líneas física de reloj y datos con seguridad. Como el paso de nivel bajo a nivel alto depende del tiempo de carga pasivo que impone la resistencia, las transiciones de voltaje son más suaves, y limitan la velocidad del bus a frecuencias estándar de 100 o 400KHz, quedando este bus destinado a sensores con frecuencias de muestreo bajas o intermedia que no saturan el bus, como por ejemplo el barómetro MS5611 empleado para calcular la altitud del cohete mediante la toma de la presión atmosférica, en el cual no se necesita una actualización de la variable tan rápida ya que la presión no cambia tan rápido, como pueden ser otras variables como las vibraciones del motor o los giros bruscos del cohete medidos por la IMU.

- **Bus SPI:**

En el apartado Connectivity activamos el periférico SPI1, (Serial Peripheral Interface), configurado en modo maestro bidireccional simultáneo (Full Duplex Master), dejando al microcontrolador con control total de la comunicación (el maestro) dándole permiso a los sensores (esclavos) cuando quiera que le transmiten sus medidas y permitiendo que la información viaje en ambos sentidos a la vez, es decir que puede enviar ordenes y recibir datos a la vez (se manda una orden por MOSI a la vez que se recibe una lectura del sensor por MISO). Para esto se reserva el pin PA5 para la señal SPI1 SCK que es la línea de reloj generada por el maestro, el pin PA6 para SPI1 MISO que es la línea por donde el sensor esclavo responde enviando datos hacia el maestro, y el pin PA7 para SPI1 MOSI que es la línea por donde el microcontrolador transmite órdenes hacia los dispositivos externos. A diferencia del bus I2C donde el voltaje sube despacio empujado por una resistencia pasiva, en SPI las salidas funcionan en modo push pull con transistores internos que inyectan 3.3V o drenan a masa de forma directa e instantánea (0V). Esto genera pulsos cuadrados limpios incluso a frecuencias de decenas de megahercios, lo que se traduce en un gran ancho de banda o capacidad de transmitir un volumen enorme de información por segundo. Esta velocidad es la que permite transferir un flujo continuo de mediciones y variables hacia la tarjeta de memoria digital MicroSD (el registrador de datos de vuelo - datalogger) sin provocar desbordamientos de memoria en el procesador, pudiendo leer a la vez a la IMU cientos de veces por segundo para saber cómo acelera y gira el cohete en cada instante, para enterarnos al instante de cualquier desvío y reaccionar sin retrasos que comprometan la estabilidad del vuelo.



Nota: Todos los archivos generados de este proyecto base (incluyendo el archivo de configuración .ioc de STM32CubeMX y el código generado con su estructura de carpetas) se encuentran disponibles dentro de la carpeta adjunta principal *Inscripción_Avionica_Software Roman_Galiana Romero*, concretamente en la subcarpeta *\Inscripción_Avionica_Software_1_CubeMx_F405*. Cabe destacar que, al tener que ampliar la configuración de pines y periféricos para los requisitos del Ejercicio 2, he adjuntado directamente la solución final completa con todo integrado. De este modo se conservan todos los elementos configurados en este primer ejercicio y simplemente se añaden los nuevos recursos sobre el mismo proyecto base.

EJERCICIO 2: Selección e integración de sensores

- a. **Propón y justifica qué sensores consideras imprescindibles en la aviónica de un cohete sonda. Elige los modelos (ejemplo: acelerómetro HLIS200), justificando por qué.**

Para que la aviónica del cohete funcione de manera robusta y tome las decisiones correctas en pleno vuelo, hace falta un conjunto de sensores capaz de aguantar el instante de despegue, los cambios de presión en la atmósfera y las maniobras de recuperación. El sistema tiene que saber en todo momento dónde está, a qué velocidad va, hacia dónde apunta y cómo acelera, además de clavar el instante exacto del apogeo para abrir el paracaídas y darnos las coordenadas para ir a recogerlo tras aterrizar.

Para organizar esto de forma limpia, la aviónica debe tener sí o sí estos tres tipos de sensores distintos:

- **Sensores de inercia y movimiento (IMU y acelerómetro High-G):** Se encarga de indicarnos la orientación del cohete en el espacio y de darnos su aceleración en tiempo real. Tiene que poder ser capaz de registrar todo el empuje del motor y los golpes secos de las separaciones pirotécnicas sin que los sensores hagan tope. Si estuviéramos diseñando un cohete pequeño para subir a 10 km, con una IMU normal bastaría para casi todo el vuelo; pero si el reto es superar la línea de Karman (> 100 km), hacen falta dos etapas, motores de alta potencia y desacoples violentos, por lo que un acelerómetro High-G capaz de medir aceleraciones del orden 100g se vuelve imprescindible para no quedarse a ciegas de lo que este haciendo el cohete en esos instantes.
- **Sensores de presión y altitud ($2 \times$ redundancia):** Dos barómetros para medir la presión estática del aire, calcular la altitud y pillar el momento justo en el que el cohete frena su subida ($v_z = 0$) para disparar el paracaídas en el punto más alto (el apogeo). Es el único sensor que propongo duplicar porque, al contrario que los demás (que van bien protegidos dentro de la bahía), los barómetros necesitan tomas de presión abiertas hacia el exterior del fuselaje, lo que los deja expuestos a taponarse con hollín del motor, polvo de la rampa o turbulencias severas al romper la barrera del sonido. Si uno se tapona o falla, el otro salva el vuelo.
- **GPS:** La antena que pilla las constelaciones para darte la posición absoluta (latitud, longitud, altitud) y velocidad en pleno vuelo (gracias al efecto Doppler), además poner en hora el reloj de la placa. No vale para disparar directamente el paracaídas por sí solo en el apogeo porque aparte de que actualiza lento, en el eje vertical mete ruido (le haría falta un buen algoritmo para corregir su medida), pero es clave para capturar la altitud a altas altitudes cuando fallan los barómetros por falta de presión atmosférica y mandar las coordenadas por telemetría para encontrar el cohete rápido nada más tocar el suelo.

Todos los chips que he elegido van en montaje superficial (SMD): apenas ocupan sitio, no añaden peso y aguantan de sobra las vibraciones y el golpe del encendido sin soltarse de la placa, ideal para un cohete sonda. En la siguiente tabla resumo los modelos elegidos, sus especificaciones reales sacadas de distribuidores oficiales y el porqué de cada uno:

TIPO	IMU (Fusión de Actitud y aceleración)	Acelerómetro High-G (Aceleración)	Baroaltímetro (2× redundancia)	GNSS / GPS (coordenadas de longitud, latitud y altitud)
MODELO	BNO085	H3LIS331DLTR (H3LIS331)	MS561101B A03-50 (MS5611)	NEO-M9N-00B (NEO-M9N)
FABRICANTE	CEVA	STMicroelectronics	TE Connectivity / Measurement Specialties	u-blox
FUNCIÓN	Estimación inercial en 9 ejes: da la aceleración lineal en los 3 ejes, velocidad angular y actitud del cohete en cuaterniones y/o ángulos de Euler (pitch, yaw y roll)	Medir la aceleración real del motor en los ejes X, Y, Z durante el despegue, separaciones de etapas y apertura del paracaídas, sin saturarse ante picos bruscos o vibraciones extremas.	Medición de presión barométrica estática para cálculo de altitud y detección de apogeo ($v_z = 0$) si es un cohete que no aspira a altitudes elevadas.	Medir la posición y velocidad en vuelo, sincronización temporal (poner en hora el reloj de la aviónica), y recuperación post-vuelo (dar las coordenadas para encontrar el cohete tras aterrizar.)
TAMAÑO / ENCAPSULADO	SMD LGA-28 ($3.8 \times 5.2 \times 1.1$ mm)	SMD TFLGA-16 ($3.0 \times 3.0 \times 1.0$ mm)	SMD QFN-8 ($3.0 \times 5.0 \times 1.0$ mm)	Módulo SMD LCC ($12.2 \times 16.0 \times 2.4$ mm)

PRECISIÓN / RESOLUCIÓN	Orientación con un error estático de 2.0° y un error dinámico de 3.5° en el cálculo de rotación (en condiciones reales precisión típica de 5°). Acelerómetro de 12 bits configurado en un rango de ± 8 g con precisión de 0.35 m/s ² . (resolución muy fina de $\approx$ 3.9 mg/digit)	Conversor A/D de 16 bits con rangos dinámicos configurables de $\pm 100g$, $\pm 200g$ y $\pm 400g$ (sensibilidad de 49 mg/digit, 98 mg/digit y 195mg/digit respectivamente).	Conversor ADC de 24 bits con resolución de 1.5 mbar ($\approx$ 10 cm de variación de altitud). Rango operativo de presión de 10mbar a 1200mbar (hasta los hasta los $\sim$ 31 km).	Precisión horizontal de 2.0 m CEP (Circular Error Probable) con recepción simultánea de 4 constelaciones (GPS, Galileo, GLONASS y BeiDou), compatible con aumentación SBAS/EGNOS.
BUS / INTERFAZ	SPI / I2C (asignado al bus SPI1 mediante CS).	SPI / I2C (asignado a bus SPI1 mediante CS).	I2C / SPI (asignado a bus I2C1)	UART / SPI / I2C / USB (conectado a canal UART con soporte DMA).
CONSUMO	Normal: $\sim$ 3.3 mA (a 3.0 V)	Ultra bajo consumo: $\sim$ 300 μ A (a 3.0 V).	Ultrabajo consumo: $\sim$ 12.5 μ A (a 3.0 V).	Moderado / Alto: 43 mA (rastreo multiconstelación continuo a 3.0 V)
PRECIO UNITARIO (1 ud.)	13,23 €	10,45 €	10,97 €	11,94 €

**Datos de especificaciones técnicas y precios unitarios extraídos directamente del distribuidor oficial Mouser Electronics y de los datasheets de los fabricantes.*

Con esta distribución de sensores, el microcontrolador recibe medidas limpias durante todo el vuelo y liberamos a la CPU de trabajo innecesario para que se enfoque en el control del cohete. El BNO085 ya le entrega los cuaterniones finales a través de su propio coprocesador, el H3LIS331DL se activa solo cuando la aceleración del motor satura a la IMU, y los dos barómetros van en el bus I2C dedicados en exclusiva a vigilar los cambios de presión para asegurar el apogeo (en un cohete que no llegue a más de 30km, pues partir de aquí tendríamos que utilizar el GPS para ello). Finalmente, reservando el bus SPI de alta velocidad para capturar la orientación del cohete en

tiempo real y aislando el GPS por su propio puerto UART independiente, garantizamos un flujo ininterrumpido de datos durante toda la trayectoria sin saturar los buses y protegiendo al bucle principal contra retrasos que comprometan el guiado del cohete.

A continuación, detallo cómo funciona cada sensor y porque he elegido a cada uno de ellos:

1. IMU BNO085

Calcular la orientación del cohete con una IMU normal suele saturar al microcontrolador principal ya que, para sacar la orientación limpia sin ruido, la CPU tendría que pasarse el vuelo haciendo multiplicaciones de matrices y corriendo filtros de Kalman pesados. El BNO085 soluciona esto porque lleva dentro su propio mini procesador de 32 bits (ARM Cortex-M0+). Este chip lee directamente los 3 ejes del acelerómetro, los 3 del giroscopio y los 3 del magnetómetro (9 ejes o 9-DOF en total), fusiona y filtra las lecturas de los 3 componentes en tiempo real por su cuenta mediante un filtro de Kalman, y le entrega al micro los ángulos y cuaterniones ya calculados, dejando a la CPU libre para otras tareas críticas.

Además, su algoritmo está preparado para no desorientarse cuando el motor pega el empujón inicial: es capaz de separar el tirón constante de la gravedad terrestre de la aceleración brutal del cohete, evitando que el cálculo de la inclinación se falsee en pleno ascenso. Al combinar los 9 ejes, el propio sensor garantiza una muy buena precisión: un error típico de 2.0° en estático y de solo 3.5° en dinámico mientras el cohete se mueve y acelera, entregando una orientación muy fiable sin que tengamos que programar la fusión matemática en el micro.

Otro motivo clave para elegirlo es que está nueva versión deja atrás los graves problemas de *clock stretching* que arrastraba su modelo anterior (el BNO055). En el protocolo I2C, cuando el procesador interno del sensor no daba abasto calculando las matrices de orientación, forzaba físicamente la línea de reloj (SCL) a nivel bajo para pausar al microcontrolador maestro mientras terminaba sus operaciones. En los microcontroladores STM32, esa pausa tan larga activaba un fallo de fábrica que tienen los chips STM32 en su circuito de I2C, haciendo que el microcontrolador se quedara pillado en un bucle infinito y congelara las comunicaciones para todo el vuelo. El BNO085 arregla esto metiendo un procesador interno ARM mucho más rápido, que tiene los datos listos al instante sin tener que frenar al micro, y además permite conectarlo directamente por bus SPI, que no tiene este problema de bloqueos y lee los datos a toda pastilla sin colgarse.

2. Acelerómetro H3LIS331DLTR

Este sensor es imprescindible si el objetivo es subir alto o buscar la línea de Kármán. Cuando el motor de combustible sólido se enciende en la rampa, mete un empujón y un golpe inicial fortísimos, a los que luego se suman las explosiones de separación de etapas y el tirón del paracaídas. Una IMU típica hace tope en el orden de $\pm 10g$ y se queda ciega ante estos picos. El H3LIS331DLTR está pensado justo para soportar grandes golpes mecánicos de hasta $10000g$ y medir aceleraciones continuas en los 3 ejes sin inmutarse, pudiendo configurarse en escalas de $\pm 100g$, $\pm 200g$ o $\pm 400g$.

Lleva un conversor interno de 16 bits que le da una gran sensibilidad en sus 3 configuraciones a pesar de tener un alto rango de medida. Al no saturarse en estas altas aceleraciones, el microcontrolador puede integrar todas las aceleraciones ($v = \int a \, dt$) y saber con precisión cuándo se apaga el motor (burnout), además de permitir comprobar si el empuje real coincide con las curvas de empuje teóricas.

3. Barómetro MS5611

El MS5611 es el barómetro estándar en cohetaría por su enorme precisión. Cuenta con un conversor de 24 bits capaz de notar cambios de presión de apenas 0.012mbar (equivalente a 10cm de altitud) lo que equivale a detectar desniveles de solo 10 cm. Gracias a su precisión permite detectar el instante exacto en el que el cohete deja de subir (si no buscamos altitudes mayores a 30000km) y su velocidad vertical se hace cero ($v_z = 0$) en el apogeo, lo que permite abrir el paracaídas justo en el punto más alto con el mínimo retraso posible.

4. GPS NEO-M9N

El módulo GPS NEO-M9N se encarga de saber la posición y velocidad en vuelo, poner en hora el reloj de la aviónica y dar las coordenadas exactas de aterrizaje para recuperar el cohete tras el vuelo. Puede conectarse a la vez a las 4 redes de satélites principales (GPS, GLONASS, Galileo y BeiDou), rastreando más de 25 satélites al mismo tiempo. Da una precisión horizontal de 2m de CEP (circular Error Probable) que básicamente significa que el 50% de las mediciones caen dentro de un círculo de radio igual a 2 metros.

Es el componente que más batería gasta de la placa ($\sim 43 \text{ mA}$), lo cual es normal porque mantiene los receptores de radio encendidos todo el tiempo leyendo las 4 constelaciones a la vez. Para no perder la conexión en pleno vuelo, monta una etapa de radiofrecuencia muy superior a la de un GPS normal basada en tres elementos:

- Filtro SAW (*Surface Acoustic Wave*): Actúa como un filtro ultraselectivo en la entrada de la antena. Toda la electrónica propia del interior del cohete (convertidores de potencia, microcontrolador y el transmisor de telemetría) genera un montón de ruido electromagnético que no permitiría leer cualquier señal débil. El filtro SAW bloquea todas esas interferencias y deja pasar únicamente la banda exacta de los satélites GNSS ($\sim 1.575 \text{ GHz}$).
- Amplificador de bajo ruido (LNA - Low-Noise Amplifier): La señal que emite un satélite a más de 20 000 km llega al fuselaje con una potencia extremadamente baja. El LNA se encarga de subirle el volumen a esa señal diminuta nada más entrar por la antena sin meterle ruido estático de fondo, permitiendo que el chip distinga los datos incluso cuando el cohete cabecea y la antena no apunta directamente al cielo.
- Oscilador compensado en temperatura (TCXO - Temperature-Compensated Crystal Oscillator): Es el reloj interno del GPS. Para calcular la posición, el chip mide diferencias de tiempo diminutas (una millonésima de segundo de error equivale a desviarse 300 metros). Como el cohete pasa del calor de la rampa al frío extremo de la atmósfera, un cristal de cuarzo común se dilataría, perdería la frecuencia y desengancharía los satélites. El TCXO mide los cambios térmicos y

autocorriga su ritmo al instante para que el reloj no se desincronice ni una milésima.

Todo este conjunto es clave para no perder la señal en el momento crítico de la apertura del paracaídas, el cohete empieza a balancearse violentamente de un lado a otro y a girar de forma caótica. Un GPS corriente perdería el enganche con los satélites y tardaría medio minuto en resincronizarse, mientras que el NEO-M9N no suelta los satélites y sigue mandando coordenadas por la radio hasta que toca el suelo.

- b. Integra en el mismo proyecto generado en el anterior ejercicio el código necesario para inicializar y leer los datos de los sensores previamente seleccionados a través de los buses anteriormente configurados.**

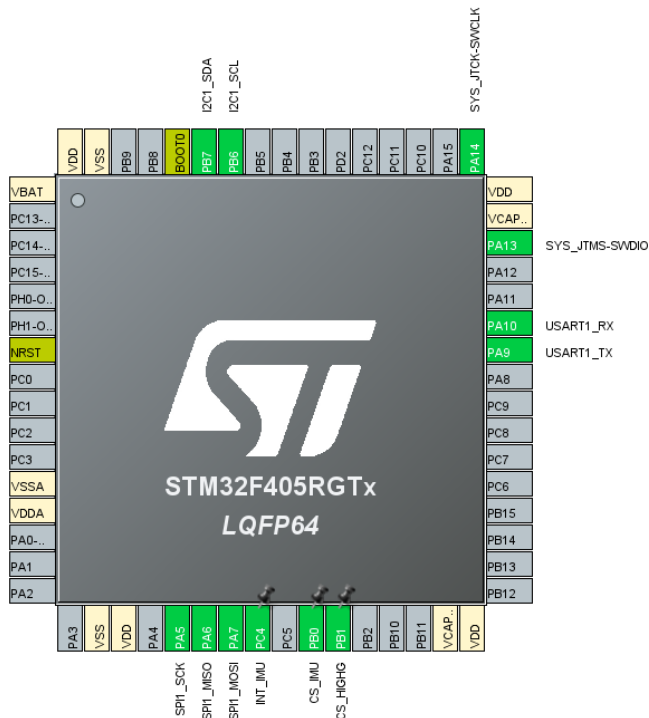
Antes de programar la inicialización y la lectura de datos de los sensores, tuve que retocar un par de cosas en CubeMX respecto a lo que tenía en el ejercicio 1 para adaptarme a lo que piden los sensores en sus hojas técnicas (data sheets)

Lo primero fue habilitar dos pines de salida digital de Chip Select para el bus SPI: PB0 como CS_IMU para la IMU BNO085 y PB1 como CS_HIGHG para el acelerómetro H3LIS331DL. Como ambos sensores comparten los mismos cables de datos (MISO y MOSI), dejamos estos dos pines en nivel alto (3.3V) por defecto. En el protocolo SPI, los chips están diseñados para escuchar solo cuando su pin de selección recibe un voltaje bajo. Por eso, cuando el microcontrolador quiere hablar con un sensor concreto, baja su pin específico a 0V, realiza la transmisión, y al terminar lo vuelve a subir a 3.3V para dejar el bus libre para el otro.

Además, para la IMU BNO085 no basta solo con el pin de selección, sino que tuve que asignar un pin de interrupción (asigne el PC4 para ello (INT_IMU)). Este sensor incorpora su propio coprocesador interno encargado de resolver la fusión sensorial y generar las tramas de orientación, requiriendo avisar activamente al microcontrolador bajando esta línea a cero voltios cada vez que tiene un paquete nuevo en su memoria interna listo para ser enviado. Si no ponemos este pin de interrupción para escucharlo a tiempo (ya que tiene una frecuencia de muestreo constante que le configuramos nosotros mismos), la memoria intermedia del sensor se satura con los siguientes cálculos, provocando que eventualmente se congele y deje de mandar datos durante todo el vuelo.

Por otro lado, tocó cambiar la configuración del periférico SPI1 porque por defecto CubeMX viene en Modo 0 y el BNO085 exige operar obligatoriamente en Modo 3. La polaridad del reloj (CPOL) define cómo está la señal de reloj cuando no se transmiten datos: en Modo 0 descansa a 0 V (CPOL=LOW), mientras que en Modo 3 descansa arriba en 3.3V (CPOL=High). Por otra parte, la fase (CPHA) define en qué cambio de voltaje lee el sensor cada bit del cable: con CPHA = 2 Edge, el chip no lee en el primer flanco de subida sino en el segundo de bajada. Si dejásemos el Modo 0, el reloj descansaría abajo y muestrearía a destiempo, provocando que el BNO085 recibiese datos desfasados y no respondiese a nada. Como el acelerómetro H3LIS331DL admite tanto el Modo 0 como el Modo 3, poner CPOL = High y CPHA = 2 Edge nos permite trabajar con ambos sensores en el mismo bus sin conflicto alguno.

El resto de los periféricos se mantienen idénticos: el bus I2C1 en PB6 y PB7 a 400kHz para alternar la lectura de los dos barómetros MS5611 mediante sus dos direcciones distintas (0x77 y 0x76), y la USART1 en PA9 y PA10 a 115200 bps (baudios - bits/s) para recibir los paquetes de telemetría del GPS NEO-M9N. Con estas correcciones aplicadas en el generador de código, pasamos directamente a programar la inicialización y la lectura de los datos de cada sensor.



Nota: Como se puede observar en la distribución de pines de la figura, esta configuración corresponde al proyecto final completo que ya adelanté en el Ejercicio 1. Al tratarse de una ampliación progresiva sobre la misma base hardware, el archivo .ioc y el código generado disponibles en \Inscripción_Avionica_Software_1_CubeMx_F405 integran tanto los periféricos iniciales como todas las nuevas líneas añadidas: el bus SPI1 compartido, las señales de Chip Select, la línea de interrupción y el puerto USART1 (en PA9 y PA10 a 115200 bps) para la telemetría del GPS, sin haber alterado nada de lo anterior.

Para programar todo esto, la idea al principio era meter el código en C puro directamente en el código del proyecto que genera CubeMX dentro de STM32CubeIDE. Pero cuando me puse a hacerlo me di cuenta del lío que era pues en C no tienes librerías masticadas como en otros sitios. Para que cada sensor funcionase tenía que buscarme la vida en GitHub, descargarme los drivers sueltos de los fabricantes (archivos .c y .h de Bosch, SparkFun, etc.), lo cual hice primero para intentarlo, meterlos a mano en las carpetas del proyecto y programar desde cero la comunicación para traducir los registros de cada sensor a las funciones HAL (hardware Abstraction Layer- capa de Abstracción de Hardware) de STM32 que son las funciones en C que proporciona STMicroelectronics para que manejar los periféricos del micro.

La verdad es que todavía no tengo tanta experiencia con C puro a tan bajo nivel como para ponerme a gestionar punteros, estructuras de registros y buffers para cinco sensores a la vez. En cambio, en C++ se algo más y tengo más experiencia y me permite utilizar librerías ya preparadas por la comunidad y los fabricantes para cada sensor, que ya gestionan toda la comunicación interna.

Así que he elegido programar el microcontrolador en C++ usando el entorno de Arduino con el paquete oficial STM32duino (el core oficial de STMicroelectronics para STM32). Con esto el código también se compila correctamente en el microcontrolador STM32F405 que configure en CubeMX. La gran ventaja de esto es que puedo usar librerías de C++ ya testeadas para leer cuaterniones, decodificar el GPS o calibrar los barómetros, mientras que por debajo el compilador traduce todo al micro respetando los pines exactos que definí. Así me ahorro pelearme con registros y cables a bajo nivel, y puedo centrarme en la física y la navegación del cohete.

Por lo tanto, para comprobar que todo el hardware responde y lee correctamente sobre la placa antes de meter la lógica pesada de vuelo, he programado un script completo en C++ (`inicializacion_lectura_sensores.cpp`) que inicializa y testea todos los sensores a la vez. Más abajo explico su estructura y las funciones clave que he utilizado.

(Todos los archivos de este programa se encuentran disponibles dentro de la carpeta general adjunta `Inscripción_Avionica_Software_Roman_Galiana_Romero`, en la subcarpeta `\inicializacion_lectura_sensores`. Dentro he incluido tanto el código fuente en formato C++ (.cpp) como el archivo de sketch listo para abrir y compilar en el entorno de Arduino (.ino), para que se pueda revisar o cargar directamente desde cualquier plataforma).

`inicializacion_lectura_sensores.cpp`

Este programa está diseñado para inicializar y validar en banco de pruebas todos los sensores seleccionados de la aviónica (los dos barómetros MS5611, la IMU BNO085, el acelerómetro High-G H3LIS331DL y el módulo GPS NEO-M9N), comprobando que cada periférico responde por su bus físico correspondiente y leyendo sus variables en tiempo real sin saturar la CPU ni el monitor serie.

Para hacer el código he decidido apoyarme en librerías probadas en C++: `MS5611.h` (de Rob Tillaart) para los barómetros, `Adafruit_BNO08x.h` para la IMU, `SparkFun_LIS331.h` para el acelerómetro de alta escala y `TinyGPS++.h` para procesar el flujo NMEA del GPS. En lugar de tener que implementar los cálculos de compensación del barómetro por ejemplo, y gestionar los búferes y protocolos de comunicación desde cero en registros de bajo nivel, estas librerías nos permiten configurar los chips en pocas líneas y extraer medidas físicas limpias de aceleración, presión y actitud.

El código se divide en dos bloques: las funciones `setup()` y `loop()`.

La función `setup()` se ejecuta una única vez al alimentar la placa y prepara los puertos y buses de comunicación:

- `Serial.begin(115200)`: Inicia la comunicación serie con el ordenador a una velocidad alta de 115.200 baudios (bit/s).
- `Wire.setSCL(PB6)`, `Wire.setSDA(PB7)` y `Wire.begin()`: Mapea e inicia el bus I2C1 en los pines específicos configurados en CubeMX, configurando la señal de reloj para transferencias estables.

- SPI.setSCLK(PA5), SPI.setMISO(PA6), SPI.setMOSI(PA7) y SPI.begin(): Asigna e inicia el bus maestro SPI1 para comunicar la IMU y el sensor High-G a alta velocidad.
- SerialGPS.begin(115200): Configura el puerto hardware USART1 asignado a PA9 y PA10 a la tasa de 115.200 baudios a la que emite el receptor NEO-M9N.
- pinMode(...) y digitalWrite(..., HIGH) en PB0 y PB1: Define los pines de Chip Select de la IMU y del acelerómetro High-G como salidas digitales, fijándolos a nivel alto (3.3V) por defecto para que la línea compartida de SPI quede en reposo y ningún sensor interfiera en el bus.
- baro1.begin() y baro2.begin(): Inicializa individualmente cada barómetro en sus direcciones I2C independientes (0x77 y 0x76). Esta función comprueba la comunicación leyendo los coeficientes de calibración de fábrica* guardados en la memoria interna (PROM - Programable Read-Only Memory) de cada sensor y si alguno no responde, el programa se detiene para avisar del fallo de hardware.

** Como ningún chip sale perfecto de fábrica, el fabricante calibra cada MS5611 en laboratorio y le deja grabados en su memoria interna (PROM) seis números fijos (C_1 a C_6). El sensor por su cuenta no da directamente unidades de presión ni de temperatura, solo escupe las lecturas digitales en bruto del convertidor (D_1 y D_2), pero como la presión cambia bastante según se caliente o se enfríe el chip, leer ese dato directo no vale para nada. Para ello se aplica la compensación que consiste en coger esos dos números crudos y los seis coeficientes, y meterlos en las fórmulas de corrección polinómica que proporciona el datasheet del MS5611 para limpiar el efecto de la temperatura y sacar la presión y los grados reales.*

- imu.begin_SPI(PB0, PC4): Inicializa la comunicación SPI con el BNO085 asignando el pin PB0 para activar el chip (CS) y el pin PC4 para escuchar la interrupción de datos listos (INT_IMU).
- imu.enableReport(...): Da la orden al procesador ARM interno del sensor para que empiece a calcular y transmitir tanto la aceleración lineal total (SH2_ACCELEROMETER) como el vector de rotación tridimensional en cuaterniones (SH2_ROTATION_VECTOR) a una frecuencia fijada de 100 Hz (cada 10000 μ s).
- highG.begin(LIS331::USE_SPI), setPowerMode(...) y setODR(...): Pone en marcha el H3LIS331DL por SPI, lo saca de su modo de bajo consumo de reposo para pasar a modo normal, sincroniza su frecuencia de muestreo (ODR) a 100 Hz y fija su rango en ± 100 g con una sensibilidad de 49 mg/dígito.

La función loop() se ejecuta de manera ininterrumpida y divide su trabajo en dos fases:

- **Vaciado asíncrono y continuo de búferes:**
 - while (SerialGPS.available() > 0): Vacía byte a byte la memoria intermedia de la UART del GPS conforme llegan los caracteres NMEA, pasándolos a

gps.encode(...) para que verifique la suma de comprobación (*checksum*) y los convierta a números reales.

- while (imu.getSensorEvent(&sensorValue)): Extrae a la máxima velocidad del microcontrolador todos los paquetes acumulados en el protocolo de transporte SHTP* del sensor, evitando que la cola interna de la IMU se desborde. También guarda la última aceleración y el último cuaternión en variables estáticas para que siempre tengamos el valor más fresco disponible.

**Nota sobre el protocolo SHTP: A diferencia de un sensor simple que solo deja lecturas fijas en memoria (en sus registros fijos) para que vayas a buscarlas, el BNO085 calcula muchas cosas a la vez (orientación, aceleraciones, calibración). Para no mezclar todos esos datos por los mismos cables del SPI, usa el protocolo SHTP (Sensor Hub Transport Protocol). Este protocolo consiste en organizar la información en paquetes, en los cuales cada dato (ya sea de aceleración u orientación) va precedido de 4 bytes: dos indican el tamaño total del paquete, otro es un contador de paquetes (1, 2, 3...) y el otro es una etiqueta con un número que le dice qué tipo de dato es (si aceleración, orientación, etc.). Gracias a esto, el micro sabe al instante a qué variable mandar los números sin mezclar los sensores y comprobar que ningún dato o paquete (gracias al contador) se ha perdido por el camino.*

- **Envío de telemetría a 4 Hz temporizado:**

En lugar de utilizar un delay(250) (lo cual congelaría el procesador durante un cuarto de segundo impidiendo que se lean los bytes del GPS y acumulando retraso), se utiliza la comprobación if (millis() - ultimoTiempo >= 250). Solo cuando transcurre ese tiempo se ejecutan las siguientes acciones:

- baro.read() y baro.getPressure(): Ordena la conversión ADC de presión y temperatura en ambos barómetros, recupera los valores en hPa y calcula la altitud mediante la fórmula del modelo ISA para la troposfera terrestre (hasta 11 km).
- highG.readAxes(...): Lee los registros crudos de 16 bits del H3LIS331DL y los transforma a Gs para yo luego transformarlos a unidades S.I. (m/s^2).
- Serial.print(...): Vuelca por pantalla las lecturas de los barómetros, la aceleración del sensor High-G, la aceleración normal y la actitud en formato de cuaternión.
- if (gps.location.isValid()): Comprueba si el receptor GPS ya tiene fijada la posición real (latitud, longitud y altura) válida, para mostrar en pantalla las coordenadas con 6 decimales de precisión, los satélites enlazados y la altitud sobre la tierra; y si todavía no tiene fijadas las coordenadas, muestra el estado de espera.

Este programa de testeo permite validar que toda la electrónica de a bordo lee y convierte bien sus magnitudes antes de volar, asegurando que los datos le lleguen limpios y sin colgarse al Filtro de Kalman y a la máquina de estados del cohete, que explicaré justo abajo en el ejercicio 3.

EJERCICIO 3: Especialización

II. En un vuelo real al espacio, ciertas variables críticas no pueden obtenerse directamente de los sensores de la aviónica. Para remediar esto, se emplean algoritmos de fusión sensorial para aprovechar los datos que sí se pueden medir y obtener los que no.

a. ¿Qué algoritmos de esta índole crees que deben implementarse en un cohete sonda que llegue al espacio?

En un vuelo real al espacio, ninguna variable crítica como la altitud, la velocidad o la orientación puede obtenerse de forma directa y fiable usando un solo sensor durante todo su vuelo. La aviónica de un cohete vuela en un entorno muy desfavorable, sometida a vibraciones extremas, cambios brutales de temperatura, regímenes supersónicos y un ambiente de bajísima densidad que roza el vacío absoluto. Si confiásemos ciegamente en las lecturas individuales de un solo sensor para su respectiva variable, el sistema de navegación colapsaría inevitablemente debido a las limitaciones físicas de cada componente. Para entender por qué es obligatorio fusionar los datos, basta con analizar qué le pasa a cada sensor cuando lo sacas de un entorno ideal y lo metes en un cohete real.

Si pensamos en los acelerómetros, la idea teórica de la navegación inercial es muy bonita para ser verdad: mides la aceleración, la integras una vez para sacar la velocidad, y la integras una segunda vez para sacar la posición. El problema está en cómo funcionan exactamente los acelerómetros MEMS (microelectromecánicos) que usamos. Estos sensores llevan una masa microscópica de silicio suspendida por muelles diminutos entre unos peines fijos. Esos dientes enfrentados forman pequeños condensadores eléctricos. Cuando el cohete acelera, la masa se desplaza por inercia y los muelles ceden, cambiando la distancia entre los dientes y alterando su capacidad eléctrica. El chip mide esa variación de capacitancia y la traduce a m/s^2 . Con el calor del sol en la rampa, la propia electrónica encendida y la fricción del aire al volar, el silicio se dilata (y además su coeficiente de dilatación varía con la temperatura, haciendo este error todavía más difícil de predecir). Al dilatarse, la distancia en reposo entre dientes cambia, cambiando con ella la capacitancia del circuito. Esto es lo que provoca el famoso sesgo térmico o sesgo residual. El sensor se cree que la masa se ha movido por inercia y te dice por ejemplo que estás acelerando a 0.05 m/s^2 cuando en realidad estás totalmente quieto. Al meter esa lectura en la doble integración, ese fallo constante se convierte en un error de posición que se dispara con el cuadrado del tiempo (t^2), haciendo inviable fiarse solo del acelerómetro. En un vuelo de apenas un par de minutos, ese error acumulado te puede hacer creer que estás a kilómetros por encima o por debajo de donde realmente te encuentras.

Por otro lado, se puede pensar que este problema de la posición que hay con el acelerómetro lo soluciona un barómetro ya que los barómetros miden la presión atmosférica para deducir la altitud. Sin embargo, este sensor también tiene sus

limitaciones. Cuando el cohete cruza la barrera del sonido, se generan ondas de choque y perturbaciones de presión brutales alrededor del fuselaje. El puerto estático por donde entra el aire deja de medir la presión atmosférica real y pasa a tragarse ruido aerodinámico y fluctuaciones de presión dinámica. Aunque pongamos redundancia, como es nuestro caso llevando dos barómetros y haciendo la media de las medidas, no solucionamos este problema aerodinámico ya que, si una onda de choque tapa los puertos estáticos, los dos chips nos van a dar exactamente la misma lectura falsa a la vez. La redundancia nos salva de que un componente se quemase o falle eléctricamente, pero no de que la física del aire mienta. Luego, como es de esperar, el barómetro tiene su propio límite físico de medida al igual que todos los sensores. Sin embargo, para un barómetro es casi inevitable ya que a medida que subimos, la atmósfera se vuelve exponencialmente menos densa, y al llegar a los 30 km de altitud, la presión cae por debajo de los 10 milibares. A partir de ahí, literalmente no hay suficientes moléculas de aire chocando contra la membrana piezorresistiva para registrar una deformación medible (que es cómo funciona los barómetros en sí). A partir de aquí el barómetro se vuelve inútil y se congela, convirtiéndose incapaz de decirnos si seguimos subiendo hacia el espacio o si ya estamos cayendo.

Finalmente, podríamos pensar que el GPS nos salva cuando el barómetro muere, pero el GPS sufre un montón durante la fase propulsada. Un módulo GPS funciona rastreando señales de radio superdébiles que vienen desde satélites a 20.000 km de distancia, utilizando bucles matemáticos internos de seguimiento de fase para engancharse a la frecuencia de la onda. Sin embargo, la aceleración brutal del cohete en el encendido cambia la velocidad tan rápido que el efecto Doppler altera la frecuencia de la señal, y los bucles internos del GPS no son lo bastante rápidos para compensarlo. Todo esto hace que el GPS pierda el enganche de los satélites, y se congele en una posición temprana de la rampa sin volver a darnos un dato real hasta que el motor se apaga, y entramos en vuelo balístico suave, dejándonos sin medidas durante los segundos más críticos del despegue.

Toda esta realidad física demuestra que, para volar un cohete sonda al espacio, no basta con tener los mejores sensores del mercado, porque todos fallan, mientan o se saturan en distintas fases del vuelo. Por ello, es matemáticamente obligatorio implementar algoritmos de fusión sensorial que aprovechen las fortalezas de cada uno para corregir los defectos de los demás.

Para ello he separado estos filtros, que se pueden implementar para fusionar las lecturas de los sensores y obtener variables fiables, en dos grupos:

- Filtros de estado de traslación (para saber dónde estás y a qué velocidad vas)
- Filtros de estimación de actitud y orientación (para saber hacia donde apuntas)

- Filtros de estado de traslación (para saber dónde estás y a qué velocidad vas)

Para resolver este desastre y obtener todas las variables de la manera más fiable posible, el algoritmo por excelencia que se implementa en la estimación de la trayectoria vertical es el Filtro de Kalman. Lo que hace este filtro es fusionar lo que creemos que está pasando según la física y lo que nos dicen los sensores, y a partir de

esa diferencia y de las medidas anteriores calcula matemáticamente cuánta fiabilidad se le puede dar a la medida del sensor respecto a lo que predecimos con la física para tener la mínima probabilidad de error. Es un proceso que se ejecuta en dos pasos continuos a lo largo de todo el vuelo: la predicción y la corrección.

En el paso de predicción, el filtro funciona a ciegas utilizando un modelo cinemático basado en las leyes de Newton. En cada intervalo de tiempo, coge la última aceleración neta medida por los acelerómetros y calcula dónde debería estar el cohete y a qué velocidad debería subir sumándole el avance temporal, y integrando a partir de los valores finales del intervalo de tiempo anterior ya corregidos por el filtro. Pero el filtro no solo calcula esos valores; también lleva la cuenta matemática de su propia incertidumbre a través de una matriz de covarianza. Como se sabe que el acelerómetro no es perfecto y que arrastra ruido físico, en cada paso de predicción ese rango de incertidumbre se va haciendo más y más grande. Por lo tanto, si dejásemos al filtro funcionando solo con este paso, estaríamos haciendo una integración inercial pura y la estimación terminaría derivando al infinito.

Aquí es donde entra la corrección. Cada vez que llega una lectura externa independiente de altitud, ya sea del barómetro a baja altitud o del GPS cuando estamos más arriba, el algoritmo compara esa medición real con la altura que él había predicho matemáticamente instantes antes. A esa diferencia numérica entre lo medido y lo estimado se le llama innovación. El filtro evalúa entonces de quién debe fiarse más en ese milisegundo exacto calculando la Ganancia de Kalman, que compara la incertidumbre de la predicción matemática, contra la varianza o ruido esperado del sensor que acaba de medir. Si el sensor es muy ruidoso, el filtro apenas le hace caso y se apoya en la física y si el sensor es de fiar, corrige la estimación acercándola a la medición.

Pero el Filtro de Kalman no solo da la posición con una gran fiabilidad, sino que también obtiene variables como la velocidad vertical real y el sesgo térmico del acelerómetro que ningún sensor puede medir directamente. El filtro es incluso capaz de deducir la velocidad a la que va el cohete sin necesidad de tener un sensor que mida directamente la velocidad, cruzando la tasa de cambio de las alturas con la aceleración. Y encima si el algoritmo detecta que, durante varios ciclos seguidos, la predicción del acelerómetro dice que estamos más arriba de lo que confirman las mediciones del barómetro o del GPS, deduce que el cohete no está acelerando tanto en la realidad, sino que la masa de silicio del sensor se ha calentado y tiene un sesgo positivo. El filtro entonces ajusta ese valor en su vector de estados, lo resta en los ciclos siguientes y limpia la deriva del sensor en tiempo real.

Si quisiéramos hacer este modelo incluso más correcto matemáticamente, en lugar de un Filtro de Kalman Lineal se implementaría un Filtro de Kalman Extendido (EKF). La diferencia es que el filtro lineal asume que las ecuaciones que describen el vuelo son líneas rectas proporcionales, pero en la realidad la física no es lineal pues la densidad del aire decae exponencialmente, la resistencia aerodinámica va con el cuadrado de la velocidad y la gravedad disminuye a medida que nos alejamos de la Tierra con el cuadrado de la distancia al centro de la tierra. Para no perderse con estas curvas, el

EKF calcula la matriz Jacobiana, que no es más que una matriz de derivadas parciales que evalúa la pendiente de las ecuaciones físicas en el punto exacto donde se encuentra el cohete en cada instante. Mientras que en el filtro lineal las matrices del modelo son fijas durante todo el lanzamiento, ya que el modelo físico ya es lineal y por lo tanto la derivada ya es una constante, en el EKF la matriz Jacobiana se va recalculando en tiempo real en cada paso de tiempo (linealizando la física respecto al punto del cohete en cada instante de reloj), adaptando la física del filtro a la no linealidad de la realidad sin alterar el algoritmo de predicción y corrección, que es lo que define a un filtro de Kalman.

- Filtros de estimación de actitud y orientación (para saber hacia dónde apuntas)

El segundo gran problema en la aviónica de nuestro cohete es saber hacia dónde está apuntando en todo momento (su orientación o actitud). Igual que pasa con la velocidad, puede que no tengas instalado en tu aviónica, ningún sensor que mida la orientación directamente (sin que la deduzca de otras medidas). Los giróscopos solo miden la velocidad a la que va girando el cohete ($\omega_x, \omega_y, \omega_z$), así que para saber los ángulos tenemos que integrar esas velocidades en el tiempo. El problema es que los giróscopos arrastran ruido y deriva térmica, y al integrarlos, el error se acumula tan rápido que en unos pocos segundos el cohete cree que apunta hacia un lado cuando en realidad va hacia otro. Además, para corregir ese desvío no podemos usar el acelerómetro buscando hacia dónde tira la gravedad terrestre, porque cuando el motor empuja hacia arriba a lo bestia, esa aceleración tapa por completo a la gravedad y el sensor se vuelve inútil para saber dónde está el suelo.

A esto se le suma un gran lío matemático si intentamos usar los ángulos estándar para representar la actitud (los ángulos de Euler: cabeceo, alabeo y guiñada). Cuando quieres pasar de las velocidades de giro que mide el giróscopo a la variación de estos ángulos en el tiempo, las ecuaciones dictan que hay que dividir por el coseno del ángulo de cabeceo. Como nuestro cohete sube prácticamente en vertical, ese ángulo es de 90° , y como el coseno de 90° es cero, la matemática intenta dividir por cero, arrojando un valor de infinito. Esto es lo que se conoce como bloqueo de cardán (*gimbal lock*), que hace que en ese punto los dos ejes de rotación restantes de alabeo y guiñada colapsen y se superpongan en la misma dirección de la línea de vuelo, dejando al software perdido sin la capacidad de distinguir que giro es cual. Para quitarnos de encima este problema se usan los cuaterniones, que son números de cuatro componentes:

$$q = \begin{bmatrix} q_0 \\ q_1 \\ q_2 \\ q_3 \end{bmatrix} = \begin{bmatrix} \cos(\alpha/2) \\ u_x \sin(\alpha/2) \\ u_y \sin(\alpha/2) \\ u_z \sin(\alpha/2) \end{bmatrix}$$

La idea del cuaternión es que en vez de hacer tres giros seguidos en los ejes X, Y y Z como en Euler, lo que hace es buscar un único eje en el espacio $u = [u_x, u_y, u_z]^T$ y rotar sobre él un ángulo α . Al trabajar con estas cuatro dimensiones, la división por cero desaparece y no hay ningún ángulo donde la matemática se rompa, sin importar si el

cohete va vertical, girando o boca abajo.

Sinceramente, más allá de entender qué representan físicamente y saber que evitan el *gimbal lock*, operar con cuaterniones matemáticamente es un mundo bastante complicado en el que todavía no tengo base teórica ni experiencia práctica. Sin embargo, sé que en el sector aeroespacial existen algoritmos específicos ya desarrollados que trabajan directamente con ellos para resolver la orientación:

- **Filtros de gradiente (Madgwick y Mahony):** Son algoritmos muy ligeros pensados para microcontroladores pequeños. En vez de meterse en operaciones matriciales pesadas, van integrando los giros del giróscopo y van corrigiendo el cuaternión poco a poco usando referencias fijas, como el campo magnético de la Tierra o la gravedad cuando el cohete aún está quieto en la rampa.
- **Filtro MEKF (Kalman Extendido Multiplicativo):** Es el algoritmo estándar que se usa en misiones espaciales y satélites. Es básicamente un Filtro de Kalman adaptado a cuaterniones para no romper sus propiedades matemáticas, separando la rotación global de los pequeños errores de ángulo para corregir la orientación y limpiar la deriva de los giróscopos de la mejor manera posible.

b. Diseña e implementa en C o C++ estos algoritmos, justificando claramente el tipo de filtro que son, las medidas usadas en cada paso, y la confianza que le das a cada una de ellas.

Para la implementación de estos algoritmos he decidido programar en C++ dentro del entorno de Arduino (usando el core para STM32), ya que es donde me siento más cómodo y tengo experiencia previa trabajando, como he mencionado anteriormente.

Primeramente, quiero destacar la división del trabajo del sistema. En mi diseño no tengo que programar los algoritmos con las complejas matemáticas de los cuaterniones ya que el sensor BNO085 es inteligente y ya lleva dentro su propio procesador ARM que ejecuta internamente el Filtro de Kalman Extendido Multiplicativo. Esto significa que la IMU resuelve la orientación 3D por su cuenta, dejando la CPU del STM32 totalmente libre para centrarse en otras tareas críticas, de cuales una de ellas es el algoritmo que he implementado yo: un Filtro de Kalman Lineal de 1D para estimar la altitud, la velocidad vertical y el sesgo térmico del acelerómetro.

El núcleo de mi código se basa en un vector de estado de tres variables: la altitud estimada (z_{est}), la velocidad vertical (v_{est}) y el sesgo o deriva del acelerómetro (b_{est}). Para que el filtro funcione, necesita saber cuánto confiamos en nuestras propias predicciones frente a lo que medimos. Esto lo defino en la Matriz de Covarianza P , que guarda la incertidumbre actual. Al arrancar en la rampa, inicializo las varianzas de altitud y velocidad con valores muy bajos ($P_{zz} = 0.1$ y $P_{vv} = 0.1$), porque sé que el cohete está quieto justo en la rampa pero inicializo la varianza del sesgo altísima ($P_{bb} = 5.0$), diciéndole al filtro que desconfíe por completo de la estimación inicial del error del chip, ya que la desconocemos, obligándole a aprenderla rápido en cuanto empiece a volar el cohete.

Para ajustar cómo se comporta el modelo matemático frente a la realidad, le decimos al filtro cuánto nos fiamos de la física y cuánto de los sensores mediante dos grupos de constantes:

- La matriz de ruido del proceso (Q): Representa cuánto dudamos de la física. Le inyecto un poco de ruido a la altitud ($Q_z = 0.001$) por las turbulencias, corrientes de aire o viento que puedan darse en cada paso de tiempo, haciendo que la integración asumiendo una aceleración constante en un instante donde se ha sufrido una racha de aire no resulte en una predicción errónea; bastante ruido a la velocidad ($Q_v = 0.01$) para absorber las vibraciones mecánicas de la combustión del motor, y un ruido pequeñísimo al sesgo ($Q_b = 0.0001$) porque asumimos que la temperatura de la placa cambia despacio (en cada paso de tiempo cambia mínimamente) (en realidad el filtro asume que cambia con un paseo aleatorio (random walk) que básicamente significa que la manera en la cambia su derivada respecto al tiempo tiene de media 0, pero asumimos que esa variación, aunque tenga de media 0, es lenta).
- La matriz de ruido de medida (R): Le doy al barómetro una varianza de $R_{\text{baro}} = 2.0 \text{ m}^2$ y al GPS una de $R_{\text{gps}} = 9.0 \text{ m}^2$. Aunque en reposo sobre una mesa el

barómetro tiene un ruido de apenas 10 cm ($\sigma \approx 0.1$ m) según su datasheet, en pleno vuelo las turbulencias, las ondas de choque y los cambios de presión dinámica alrededor del fuselaje meten fluctuaciones de más de un metro ($\sigma \approx 1.4$ m, que al cuadrado da ≈ 2.0 m²), así que le subo la varianza para que el filtro tenga en cuenta ese ruido del aire. En cambio, el GPS marca en su datasheet un error de 2 m CEP en horizontal, pero por la propia geometría de los satélites (todos arriba y ninguno debajo de la Tierra) la dispersión en vertical siempre empeora hasta unas 1.5 veces el CEP, lo que en este caso supone unos $\sigma \approx 3$ m ($\sigma^2 \approx 9.0$ m²). Al poner $R_{\text{baro}} < R_{\text{gps}}$, obligamos al filtro a seguir los cambios rápidos y suaves con el barómetro, usando el GPS como un ancla a largo plazo para que la altura no derive.

El código lo he hecho teniendo en cuenta los umbrales físicos del acelerómetro y del barómetro los cuales he sacado de sus respectivos datasheets para así evitar que mientan o hagan colapsar el algoritmo.

El BNO085 tiene un rango de medida máximo de ± 8 g por lo que si usamos su acelerómetro durante el despegue, se saturará y el filtro se irá al infinito. Por eso, he programado que en cuanto la aceleración supera los 7 g ($7.0 \cdot 9.80665$ m/s²) el filtro pasa instantáneamente a recibir la lectura del acelerómetro externo H3LIS331DL, que soporta hasta 100 g (en la configuración que le he indicado yo) dejando además 1 g de margen de seguridad para evitar que la IMU se sature.

También, según el datasheet del MS5611, el sensor se vuelve incapaz de medir correctamente por debajo de una presión de 10 mbar, que en la atmósfera estándar (ISA) equivale a unos 31 056 m de altura. En el código he implementado un límite de 30 000 m, para que a partir de esa altitud el código deje de leer los barómetros y se fie solo del GPS para evitar que los barómetros inyecten ruido ciego al Filtro de Kalman.

Luego en cuanto a los dos pasos del filtro de Kalman anteriormente explicados, se ejecutan en el bucle principal a la máxima velocidad posible ajustado por el paso de tiempo (variable dt). Primero hace la predicción, sumando la aceleración limpia ($a_{\text{net}} = a_{\text{raw}} - 9.80665 - b_{\text{est}}$) para sacar velocidad y altura, y propagando la matriz en el paso del tiempo, para ver cómo crece la incertidumbre. Un detalle importante aquí es que aplico la resta con una gravedad constante de (9.80665 m/s²) el cual se podría pensar que destruye mi filtro, ya que la gravedad disminuye con la altitud (se podría programar la ley de gravitación de Newton recalculando g en cada ciclo). Sin embargo, a 100 km de altura, usar la gravedad del nivel del mar introduce un error de apenas 0.3 m/s² (un 3%). Sin embargo, como tenemos un estado dedicado a estimar el sesgo residual (b_{est}), cuando el GPS corrige la altitud a alta altitud, el filtro detecta esa diferencia de 0.3 m/s² y absorbe automáticamente ese pequeño error integrándolo dentro del término de sesgo (cree que el sensor se ha desviado un poco más) y lo compensa en tiempo real, sin que haga falta integrarlo en el código.

Luego viene el segundo paso, la etapa de corrección, que la he dejado programada a través de la función `kalman_update_altitude()` para llamarla cómodamente cada vez que entra un dato nuevo de altura. Lo primero que hace esta función es calcular la innovación (la resta entre lo que mide el sensor y lo que predecía el filtro), y mete un filtro de rechazo de valores atípicos (outliers): si la medida se desvía más de 4

desviaciones típicas ($\pm 4\sigma$) de lo esperado, directamente la descarta para que una lectura corrupta no rompa el filtro. Si pasa el filtro, calcula la ganancia de Kalman K , corrige los tres estados (altura, velocidad y sesgo) y actualiza la matriz de covarianza para reducir la incertidumbre.

En el bucle de vuelo, esta corrección se alimenta de dos sitios:

- De los dos barómetros, que como ya he dicho anteriormente, debido a su límite de medición, solo leen por debajo de 30 km. Si ambos devuelven lecturas válidas, calcula sus alturas con mi función ISA, saca la media y llama a la corrección pasándole la mitad de varianza ($0.5 \cdot R_{\text{baro}}$) porque al promediar dos sensores el ruido baja a la mitad. Si uno de los dos falla, se descarta el barómetro que ha dado problemas y el filtro corrige usando solo el sensor sano con su varianza normal (R_{baro}).
- Del GPS, que va por libre a su ritmo asíncrono. Cada vez que el módulo completa una trama NMEA con solución 3D válida, saca su altitud y llama a esta misma función de actualización pero con la varianza R_{gps} para corregir a largo plazo. Si los dos barómetros se mueren o cruzamos los 30 km, el filtro no se queda a ciegas porque sigue corrigiéndose con estas posiciones del GPS.

Y ya por último y para que el cohete sepa qué tiene que hacer con todos estos datos limpios, he metido una máquina de estados. El cohete empieza en estado PAD (en la rampa) y solo pasa a BOOST (despegue) cuando el filtro de Kalman le confirma que la velocidad es mayor de 10 m/s y que ya ha subido 15 metros, y así evitamos que un golpe fuerte en la rampa dispare el sistema por accidente. Luego, cuando el motor se apaga y el cohete deja de empujar (la aceleración neta cae por debajo de cero ya que el cohete empieza a disminuir su velocidad por la gravedad y drag aerodinámico), confirmamos el fin de la combustión (burnout) y pasamos a COAST (vuelo inercial o balístico). Aquí el cohete sigue subiendo por pura inercia hasta que la velocidad vertical calculada por el filtro cruza a cero sabiendo que en ese milisegundo exacto sabemos que hemos llegado al punto más alto de la trayectoria (el apogeo). Ahí el código da la orden de desplegar el paracaídas y pasamos al estado final de DESCENT para la caída.

Ahora que ya he explicado el funcionamiento general de mi algoritmo, paso a detallar el código en sí, donde implemento toda esta arquitectura de filtrado, redundancia sensorial y control por eventos.

(El código completo de esta lógica principal se encuentra disponible en la carpeta adjunta Inscripción_Avionica_Software Roman_Galiana Romero, dentro de la subcarpeta \main. Al igual que con el script de testeo de sensores, he incluido tanto el código fuente en C++ (main.cpp) como la versión en formato de sketch de Arduino (.ino) para facilitar su apertura y revisión en cualquier entorno).

main.cpp

Para el algoritmo de control y navegación de vuelo con fusión sensorial, he implementado un programa principal (main.cpp) que integra la lectura en tiempo real

de todos los sensores físicos, ejecuta el Filtro de Kalman Lineal de 1D con estimación de sesgo en el eje vertical y gobierna las etapas del vuelo mediante una máquina de estados protegida contra transitorios.

El código se divide en dos bloques principales:

La función `setup()` se ejecuta una sola vez al arrancar para inicializar todos los sensores y los buses de comunicación:

- `Serial.begin(...)`, `Wire.begin(...)` y `SPI.begin(...)`: Configuran la comunicación serie para telemetría (115.200 baudios), el bus I2C a 400kHz para los barómetros, y el bus maestro SPI para los sensores inerciales.
- Configuración de sensores: Arranca los barómetros MS5611, la IMU BNO085 y el acelerómetro H3LIS331DL, fijando sus frecuencias de muestreo a 100 Hz.
- Condición de aborto en rampa: Si algún sensor falla al iniciar, el micro entra en un bucle infinito (`while(1)`) y detiene la ejecución para no lanzar el cohete a ciegas.
- Calibración en rampa (`z_ground`): Se toma una batería de medidas de los dos barómetros para promediarlas con el modelo ISA y fijar la altitud de partida.

La función `loop()` es el bucle de vuelo en tiempo real que corre a máxima frecuencia realizando las siguientes tareas clave:

- Cálculo del intervalo temporal (`dt`): Mide los microsegundos entre ciclos para la integración. Se incluye un seguro numérico que fuerza $dt = 0.01$ s si el bucle sufre una anomalía de tiempo, protegiendo al filtro de Kalman de divergencias si hay bloqueos en los buses.
- Vaciado asíncrono continuo: Extrae a la máxima velocidad todos los paquetes acumulados en el protocolo SHTP de la IMU y los bytes del UART del GPS (`while (SerialGPS.available())`) para no arrastrar latencia y tener siempre el dato más fresco.
- Selección de sensor de aceleración: Conmuta al acelerómetro High-G de forma instantánea si la aceleración supera el umbral de $\pm 7g$, evitando que la IMU saturate durante los picos de empuje del motor.
- Filtro de Kalman: Ejecuta matemáticamente los dos pasos del filtro:
 - Predicción: Integra las leyes de Newton propagando el vector de estado $[z_{est}, v_{est}, b_{est}]^T$ con la aceleración limpia y aumenta la covarianza.
 - Corrección (Barómetros y GPS): Por debajo de 30 km, promedia ambos barómetros para reducir la varianza. Cuando el GPS obtiene señal, introduce su altitud para anclar la trayectoria y calibrar el sesgo térmico del acelerómetro en tiempo real.
- Volcado de telemetría: Imprime las variables filtradas, el cuaternión y el GPS a una tasa regulada de 10 Hz (usando `millis()`) sin bloquear la CPU con funciones `delay()`.

Máquina de estados (switch)

Cada casilla del switch define las reglas para pasar de una fase de vuelo a la siguiente usando las variables filtradas, exigiendo confirmación para evitar disparos en falso por ruido o vibraciones:

- De RAMPA (PAD) a FASE PROPULSADA (BOOST): La transición se activa mediante una doble condición de seguridad: el filtro debe estimar simultáneamente una velocidad superior a 10 m/s y haber ganado más de 15 m sobre la altitud de la rampa. De este modo se descarta por completo un falso despegue por un golpe, manipulación en rampa o ráfagas de viento.
- De FASE PROPULSADA a VUELO INERCIAL (COAST): Detecta el apagado del motor (*burnout*). Cuando se agota el combustible, la aceleración neta cae por debajo de cero ($a_{\text{net}} < 0$). Se exige que esta condición se mantenga durante 3 ciclos seguidos (30 ms) para no confundir ruidos de combustión con el apagado real.
- De VUELO INERCIAL a APOGEO (APOGEE): Detecta el punto más alto de la trayectoria. Al usar Kalman, no tenemos que esperar a que el barómetro baje unos metros (lo que retrasaría la apertura). El apogeo se marca en el milisegundo exacto en que la velocidad vertical cruza a cero ($v \leq 0$). Requiere 3 ciclos de confirmación y estar por encima de los 100 m.
- De APOGEO a DESCENSO (DESCENT): Es un estado de acción instantánea que dispara el paracaídas y conmuta a la fase de caída en el siguiente ciclo.
- DESCENSO: El cohete cae hacia la superficie terrestre y sistema se centra en transmitir las coordenadas GPS (latitud y longitud con 6 decimales) a la estación de tierra para localizar el impacto y proceder a la recuperación.

Se puede observar que en el enum de los estados he dejado puesto LANDED, aunque al final no he metido su casilla en el switch para no alargar más el código. Para añadirlo bastaría con mirar que la velocidad vertical que da Kalman se quede en cero ($v_{\text{est}} \approx 0$), que la altura ya no cambie y que las coordenadas del GPS no se muevan durante unos segundos. Cuando se cumpla eso, el cohete sabe que ya está en el suelo y pasa a LANDED para cortar cálculos que ya no hacen falta, pitar para que lo encontremos y seguir mandando la ubicación por telemetría para ir a buscarlo.

En esa misma línea de simplificación, el módulo GPS también entrega la velocidad midiendo el efecto Doppler de sus señales. Por no complicar el código he preferido no meter esa medida en la corrección de Kalman y dejar que el filtro calcule la velocidad por su cuenta a partir de la aceleración y la altura, aunque en una versión futura se podría extraer `gps.speed.mps()` para tener una comprobación directa de la velocidad una vez pasada la fase propulsada (ya que durante los primeros segundos de empuje violento el GPS no se suele dar datos fiables de velocidad).

Consideraciones sobre la orientación física de la placa (PCB) y la proyección inercial

Para este código he adoptado un par de simplificaciones sobre cómo va colocada la electrónica y cómo se mueve el cohete en el aire. Así es como lo he dejado programado y cómo se montaría en un lanzamiento real:

1. Posición de la PCB en el fuselaje

- En mi código: He supuesto un montaje horizontal sobre un mamparo transversal (bulkhead), mirando hacia la ojiva. Así, el eje perpendicular de los chips (el eje Z) coincide directamente con el eje axial del cohete, y el empuje entra limpio por la variable z sin tener que reasignar nada.
- En un cohete real: En cohetes de fuselajes estrechos (de 40 a 60 mm), la placa normalmente no cabe cruzada y hay que montarla en vertical a lo largo del fuselaje. Ahí el eje axial del cohete pasaría a ser el X o el Y del chip. Para solucionarlo en la integración final, basta con poner el cohete de pie en el banco de pruebas, mirar qué eje marca +1g (+9.81 m/s²) en reposo y usar esa componente como aceleración axial.

2. Dinámica de vuelo y cabeceo hacia el apogeo

- En mi código: Asumo un vuelo puramente vertical (1D) donde el cohete no se desvía de la vertical local. Por eso le resto los 9.80665 m/s² de la gravedad directamente al eje Z del acelerómetro:

$$a_{\text{net}} = a_{\text{raw}} - 9.80665 \text{ m/s}^2$$

- En la realidad: El cohete nunca sube totalmente recto. El viento, las ráfagas y la propia gravedad (gravity turn) lo van inclinando, de modo que en el apogeo el morro suele tener un ángulo apreciable. Si restamos la gravedad directamente a la lectura del sensor cuando va inclinado, metemos un error en la aceleración vertical neta porque la gravedad tira hacia el centro de la Tierra y el chip mide según apunta el cohete.

Por lo tanto para solucionar esto y calcular la altitud y la velocidad vertical exactas respecto al suelo, lo riguroso es proyectar las medidas del cohete al sistema inercial terrestre con ayuda de los cuaterniones que dicen tu orientación respecto a la tierra. Para ello se procedería de la siguiente manera:

1. Medimos la aceleración (a_x, a_y, a_z) en los 3 ejes del cohete (ejes cuerpo)
2. Con el cuaternión $q = [q_r, q_i, q_j, q_k]$ que calcula la IMU BNO085, proyectamos la aceleración total sobre la vertical terrestre (Z_{inercial}) usando la tercera fila de la matriz de rotación:

$$a_{z,\text{inercial}} = 2(q_i q_k - q_j q_r) a_x + 2(q_j q_k + q_i q_r) a_y + (1 - 2(q_i^2 + q_j^2)) a_z *$$

**Esta fórmula la he buscado y sé que es completamente válida, pero desconozco de sus principios y de cómo los quaterniones acaban en la matriz de rotación, etc. (Lo cual es una de las razones por la cual no lo he integrado en mi código.)*

3. A partir de ese valor proyectado en la vertical es al que se le resta la gravedad para calcular la aceleración vertical neta:

$$a_{\text{net, vertical}} = a_{\text{z, inercial}} - 9.80665 \text{ m/s}^2$$

Al meter esta aceleración corregida al filtro, la velocidad vertical y el instante de apogeo salen clavadas, sin importar cuánto cabeceé el cohete.

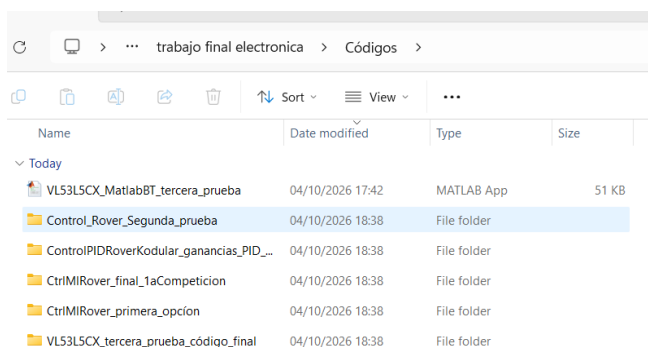
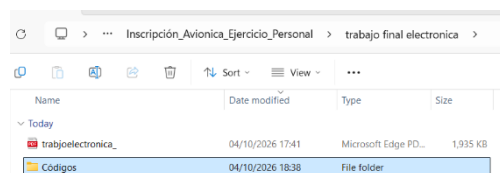
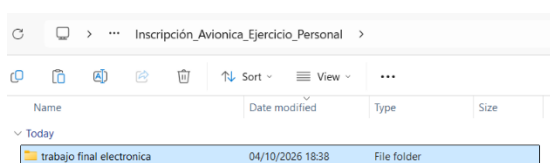
He preferido dejar la proyección 3D fuera de este primer desarrollo porque el objetivo principal era hacer un algoritmo de fusión de sensores para obtener variables que no se puedan medir directamente o para aumentar la fiabilidad de las que sí, y que el código funcione fluido, sin quedarse colgado en ningún momento, lo cual pienso que he conseguido con mi modelo 1D. Además, prefiero no meter fórmulas en mi código sin entender ni ver de dónde salen. Como el hardware ya extrae y almacena en cada ciclo las 4 componentes del cuaternión (quat_r, quat_i, quat_j, quat_k), dar el salto a un modelo 3D (de 6-DOF) sería inmediato, solo habría que cambiar la aceleración axial en bruto por la componente vertical proyectada mediante cuaterniones en la etapa de predicción.

EJERCICIO PERSONAL

Detalla algún proyecto (o varios) de software / electrónica / hardware / antenas... que hayas realizado. Puedes incluir código, programas, fotos, vídeos, y todo lo que consideres necesario. Justifica para qué lo hiciste y qué conseguiste con ello.

Para este ejercicio personal, voy a detallar el desarrollo de un rover autónomo con tracción de orugas controlado por un microcontrolador ESP32 de 32 bits, un proyecto que realicé con un compañero durante la asignatura de Tecnología Electrónica.

Antes de entrar a explicar mi proyecto, quiero comentar que he adjuntado el PDF de la memoria final del trabajo en la carpeta de este ejercicio personal, y que el archivo ZIP completo con el repositorio de GitHub va también adjunto al correo junto con los ejercicios hechos. En las capturas de la distribución de la carpeta de este ejercicio que os he ajuntado aquí bajo, podéis ver todos los archivos del proyecto, pero quiero destacar que los verdaderamente importantes y definitivos son los correspondientes a la prueba 1 (CtrlMIrover_final_1aCompeticion), la prueba 2 (Control_Rover_Segunda_prueba) y la prueba 3 (VL53L5CX_tercera_prueba_código_final y su correspondiente interfaz VL53L5CX_MatlabBT_tercera_prueba). También he incluido un script (ControlPIDRoverKodular_ganancias_PID...) que utilicé exclusivamente para el ajuste del control (con su correspondiente app Kodular que también programé).



El objetivo principal de este proyecto no era quedarse en la teoría, sino enfrentarse al comportamiento no ideal del hardware físico real (rozamientos asimétricos de las orugas, ruido en los sensores, latencia en las comunicaciones) y solucionarlo íntegramente mediante software.

Para ello, la selección de hardware fue crítica:

- **El microcontrolador:** Utilizamos el ESP32, elegido por tener conectividad Bluetooth integrada y una potencia de cálculo de 32 bits necesaria para mover matrices y algoritmos en tiempo real.
- **Visión 3D (ToF VL53L5CX):** En lugar de usar un sensor de ultrasonidos básico, integramos un láser de tiempo de vuelo capaz de devolver una matriz de distancias de 8x8 zonas, lo que nos permitía generar un mapa de profundidad del entorno (básicamente media la distancia de lo que tenía delante midiendo el tiempo en el que tardaba el laser en ir y volver, pudiendo medirlas en múltiples zonas a la vez).
- **Orientación (IMU BNO055):** Este sensor fue fundamental porque incorpora su propio coprocesador interno al igual que el BNO085 (es básicamente su modelo anterior). Este chip realiza la fusión sensorial de los acelerómetros, giroscopios y magnetómetros por su cuenta y entrega directamente los ángulos de Euler, liberando de esa inmensa carga matemática a la CPU del ESP32.
- **Actuadores y Potencia:** Para realizar el movimiento físico del rover, la etapa de potencia consistió en un driver de motores L298N (un controlador de doble puente en H). Este módulo era el encargado de gobernar la dirección y velocidad de los motores DC de las orugas de forma independiente, alimentado todo por una batería LiPo de 3 celdas (11.1V y 2200mAh).

El desarrollo y lo que conseguimos con él se estructuró a través de tres fases o pruebas de dificultad incremental:

Fase 1: Control manual y el problema humano La primera prueba consistía en implementar un control manual para recorrer un circuito superando obstáculos en el menor tiempo posible. Para ello, diseñamos una aplicación en Kodular conectada por Bluetooth. Al principio, intentamos controlar el rover con dos barras deslizantes independientes (una para cada oruga), pero físicamente era difícilísimo coordinarlas para mantener el vehículo recto.

Este es el código del script C++ de Arduino que programamos en el micro y la interfaz de la app kodular (junto a su código) utilizada para controlarlo en esa primera prueba:

```

void loop() {
  int DutyLeft, DutyRight;
  unsigned int cTonLeft, cTonRight;

  if (SerialBT.available() >= 2) {
    DutyLeft = SerialBT.read() - 100;
    DutyRight = SerialBT.read() - 100;

    if (DutyLeft > 0) {
      digitalWrite(IN1, LOW);
      digitalWrite(IN2, HIGH);
    } else if (DutyLeft < 0) {
      digitalWrite(IN1, HIGH);
      digitalWrite(IN2, LOW);
    } else {
      digitalWrite(IN1, LOW);
      digitalWrite(IN2, LOW);
    }

    if (DutyRight > 0) {
      digitalWrite(IN3, HIGH);
      digitalWrite(IN4, LOW);
    } else if (DutyRight < 0) {
      digitalWrite(IN3, LOW);
      digitalWrite(IN4, HIGH);
    } else {
      digitalWrite(IN3, LOW);
      digitalWrite(IN4, LOW);
    }

    cTonLeft = abs(DutyLeft) * 32767 / 100;
    cTonRight = abs(DutyRight) * 32767 / 100;

    ledcWrite(ENA, cTonLeft);
    ledcWrite(ENB, cTonRight);
  }
}

```

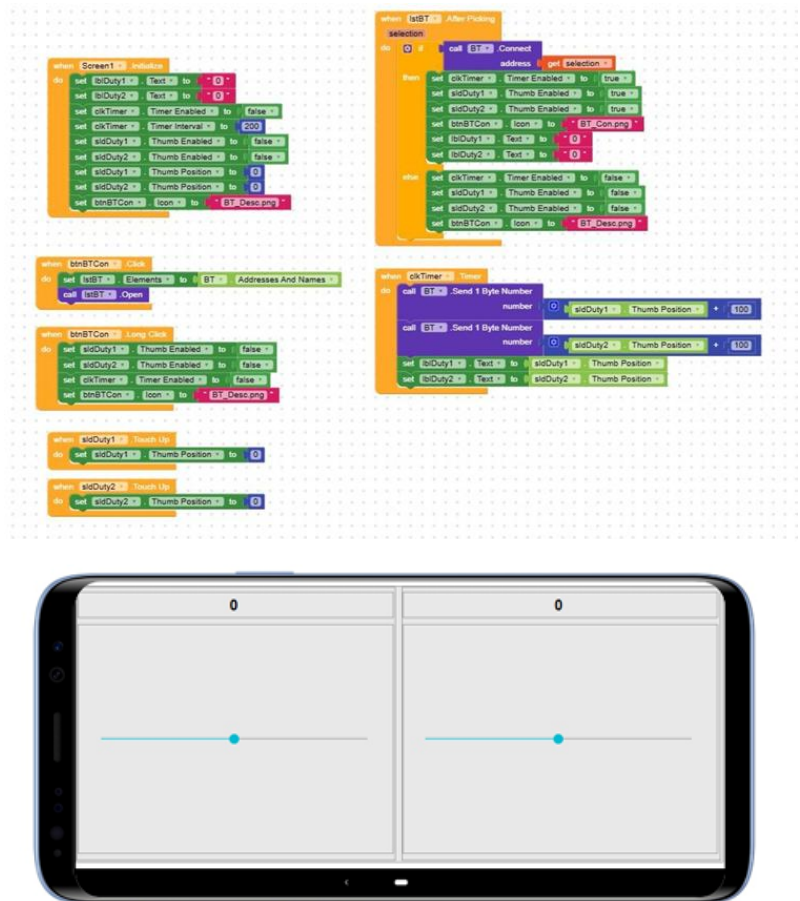


Figure 3. Interfaz de la aplicación móvil desarrollada en Kodular, correspondiente al primer intento de la primera competición.

El verdadero logro de esta fase fue rediseñar el control hacia un sistema de tracción diferencial (skid-steer)*. Modifiqué la aplicación para que solo enviara dos comandos globales: potencia longitudinal (avance o retroceso) y dirección de rotación. El código del ESP32 (CtrlMIRover_final_1aCompeticion) se encargaba de hacer la mezcla matemática por debajo, repartiendo el ancho de pulso (PWM) correspondiente a cada oruga, logrando un manejo súper intuitivo y fluido.

* Skid-steer (tracción diferencial) es una configuración donde el vehículo no tiene un eje físico para torcer las ruedas como haría un coche normal. En su lugar, el rover gira haciendo que la oruga de un lado se mueva más rápido, más despacio o en sentido opuesto a la del otro lado. En el código, esto se calcula mezclando la orden de aceleración con la orden de giro: al sumarlas para un motor y restarlas para el otro, logramos maniobras extremadamente ágiles, permitiendo incluso que el vehículo rote sobre su propio centro sin moverse del sitio (haciendo avanzar una oruga y retroceder la opuesta a la misma velocidad).

Fase 2: Conducción autónoma en línea recta (El PID) La segunda prueba nos exigía que el rover navegara de forma autónoma en línea recta por un pasillo sin chocar. En la realidad física, aunque inyectes exactamente el mismo PWM a los dos motores, el rover siempre acaba desviándose debido a las tolerancias mecánicas, el rozamiento asimétrico de las cadenas con el suelo y diferencias en la transmisión. Para

solucionar este comportamiento no ideal, implementé un control PID en lazo cerrado realimentado continuamente con el ángulo de yaw (guiñada) que proporcionaba la IMU. Si el morro del rover se desviaba de su rumbo de referencia, el algoritmo calculaba el error angular y aplicaba automáticamente una corrección diferencial en las orugas, acelerando una y frenando la otra para enderezarlo sin llegar a detener su avance. Para afinar este algoritmo de forma rigurosa, desarrollé el script `ControlPIDRoverKodular_ganancias_PID_experimentales` (junto a su app correspondiente en Kodular). Gracias a ella, pude ajustar empíricamente las ganancias proporcional, integral y derivativa en tiempo real por Bluetooth. Fui inyectando los valores mientras veía la reacción física del rover, asegurándome de que siguiera la referencia sin sobreoscilaciones ni vibraciones erráticas. Tras este ensayo, el ajuste óptimo definitivo prescindí de la ganancia derivativa por su sensibilidad a amplificar el ruido del sensor (fijando $K_p=7.5$, $K_i=6.0$, $K_d=0.0$) e incluyó límites de saturación (anti-windup) para que la memoria integral no desestabilizara el sistema.

Para la implementación del control PID se desarrolló un código en Arduino (`Control_Rover_Segunda_prueba`) que ejecutaba el lazo de control completo. El programa leía continuamente la orientación (yaw) proporcionada por la IMU BNO055 y calculaba el error angular respecto a la referencia establecida, asegurándose de mantenerlo siempre en el rango de -180 a $+180$ grados. Con este error se calculaban los términos del PID, limitando tanto el término integral como la acción de control total (saturación o anti-windup) para evitar inestabilidades. Finalmente, la acción de control se sumaba a la velocidad base de una oruga y se restaba a la de la otra, aplicando las señales PWM correspondientes para corregir la trayectoria.

Además del código en el microcontrolador, se creó una aplicación complementaria en Kodular para gestionar el sistema de forma remota por Bluetooth. Esta app contaba con un comando de "Start/Stop" que permitía activar o desactivar los motores de forma segura, y un comando de "Reset" que era clave para el funcionamiento pues al pulsarlo, el código capturaba la orientación actual del rover en ese preciso instante y la fijaba como la nueva referencia angular (0°). Esto resultaba especialmente útil ya que permitía recolocar el vehículo manualmente en el pasillo y establecer un nuevo rumbo de referencia sin necesidad de reiniciar el microcontrolador.

Fase 3: Autonomía total y evasión de obstáculos La tercera prueba era el reto definitivo: recorrer un pasillo de forma totalmente autónoma detectando y esquivando obstáculos en el camino. Si hubiéramos intentado que el ESP32 leyese y procesara la matriz 8×8 del láser, tomara decisiones de esquiva y además ejecutara el control PID de los motores a alta frecuencia, la CPU se habría saturado y el sistema habría colapsado. Por lo tanto para evitar esto la arquitectura del sistema se dividió en dos niveles para no saturar al microcontrolador:

- **El ESP32 (bajo nivel):** Actuaba como los 'reflejos' del rover. El código `VL53L5CX_tercera_prueba_código_final` se dedicaba únicamente al trabajo

físico y rápido: leer la matriz de distancias en bruto del láser ToF, leer la orientación de la IMU y ejecutar el control PID de los motores para que el vehículo avanzara recto. Para poder procesar esto, empaquetaba rígidamente la matriz de distancias en tramas de 160 bytes y las enviaba a MATLAB por Bluetooth de forma síncrona. Sin embargo, el ESP32 no sabía por qué giraba, solo cumplía órdenes.

- **MATLAB en el PC (alto nivel):** La aplicación VL53L5CX_MatlabBT_tercera_prueba hacía de "cerebro" que tomaba las decisiones de navegación. Recibía la matriz de datos por Bluetooth, la limpiaba de ruido mediante el filtro estadístico basado en la Desviación Absoluta de la Mediana (MAD)* y descartaba los rebotes erráticos del láser infrarrojo. Era MATLAB quien analizaba el pasillo, evaluaba las columnas y buscaba la ruta más libre. Al detectar un obstáculo, calculaba la maniobra y enviaba comandos comprimidos de 2 bytes al ESP32 con la velocidad adaptativa y la orden de giro relativa (por ejemplo: "reduce la velocidad y gira 17 grados a la derecha porque es la vía más libre"). Una vez superada la caja y con vía libre frontal, MATLAB le mandaba la orden de retornar a su rumbo inicial (para que se mantuviera centrado y no se chocara contra la pared). El ESP32 simplemente traducía esa orden, actualizaba la referencia angular en el PID y ejecutaba la corrección en los motores con suavidad.

**El filtrado MAD (Desviación Absoluta de la Mediana) es un método estadístico que funciona básicamente como un "detector de mentiras" para el sensor láser. Desafortunadamente, a veces el láser rebota en una superficie brillante o con un mal ángulo y devuelve una distancia absurda (por ejemplo, lee 3 metros de distancia donde en realidad hay una caja a 20 centímetros). Si usáramos un promedio matemático normal (una media), ese único valor falso distorsionaría el cálculo y el rover acabaría chocando. El filtro MAD soluciona esto fijándose en el valor central (la mediana) de las mediciones. Si detecta un punto que se desvía exageradamente de la normalidad de ese grupo, lo identifica como una medida atípica (un outlier) y lo descarta por completo antes de calcular la distancia definitiva, garantizando que el rover solo reaccione ante obstáculos reales.*

Aquí abajo detallo cosas importantes de mi código, aunque como he comentado antes el script completo del Matlab y código principal están en la carpeta adjunta.

Para alimentar a MATLAB con los datos del entorno, programé la función `data_acquisition()` en el ESP32. Su trabajo es leer el sensor láser, pero en lugar de coger la matriz entera de 8x8, la recorté para quedarme solo con una "ventana" central útil de 5x8 (leyendo desde el índice 8 al 47). Hice esto porque las filas inferiores se chocaban con el suelo y la superior solía dar medidas ruidosas, así que era información inútil que solo estorbaba para esquivar cajas. Como la distancia en milímetros casi siempre es mayor a 255 (que es el límite de lo que cabe en un solo byte), el código "rompe" matemáticamente cada medición en dos pedazos usando

divisiones y restos ($/ 256$ y $\% 256$). Hago exactamente lo mismo con los "estados" (target_status), que son básicamente un indicador de calidad que nos da el sensor para cada celda; nos dicen si la distancia que acaba de medir es totalmente fiable o si es una lectura dudosa provocada por una mala reflexión o falta de luz. Al final, el código coge esas 40 distancias y sus 40 estados de calidad, los encaja en un único tren de 160 bytes exactos, y lo dispara de golpe por Bluetooth con un solo SerialBT.write. De esta forma, el envío es rapidísimo, estructurado y evitamos atascar la conexión inalámbrica mandando textos o numeritos sueltos.

Una vez que esos 160 bytes llegan de golpe al PC, MATLAB entra en acción con la función Funcion_Recepcion para rearmar el puzzle. El código va juntando los bytes de dos en dos (multiplicando por 256 y sumando) para recuperar el valor real de las distancias y los estados. Con los estados hace un filtro rápido: si el sensor dice que es un '5' (una lectura totalmente fiable), le asigna un valor alto (1000), y si no, lo pone a cero, para saber de un vistazo qué medidas valen y cuáles no.

Después, coge esa lista larga de 40 números y le da forma de cuadrícula de 5x8 (reshape). Como la aplicación de MATLAB nos permitía visualizar este mapa de distancias en la pantalla en tiempo real, era imprescindible transponer y rotar esta matriz (rot90). Si no, la cuadrícula aparecería invertida como en un espejo, mostrando lo que estaba a la derecha en la parte izquierda y viceversa. Al enderezarla matemáticamente, nos aseguramos de que lo que se visualiza en la interfaz coincida exactamente con la posición real de los obstáculos que el coche tiene delante. Por último, el código capta cualquier distancia que pase de 800 mm; todo lo que esté más lejos de eso se ignora porque es ruido lejano que no nos interesa para esquivar la caja de turno.

Aquí abajo adjunto los dos fragmentos de código donde ocurre todo esto: a la izquierda, la función del ESP32 que recorta y empaqueta la matriz en 160 bytes; y a la derecha, la función de MATLAB que recibe ese tren de datos por Bluetooth, lo rearma, filtra los estados y endereza la cuadrícula para que coincida con la visión física del rover

```
// ----- ToF: enviar 5x8 a Matlab -----
void data_acquisition(){
  if (myImager.getRangingData(&measurementData)){
    int j = 0;
    for (int i = 8; i <= 47; i++) {
      int d = (int)measurementData.distance_mm[i];
      datos[2*j] = (byte)(d / 256);
      datos[2*j + 1] = (byte)(d % 256);

      int st = (int)measurementData.target_status[i];
      datos[80 + 2*j] = (byte)(st / 256);
      datos[80 + 2*j + 1] = (byte)(st % 256);

      j++;
    }
    SerialBT.write(datos,160);
  }
}
```

Figure 14. Selección de celdas útiles.

```
function Funcion_Recepcion(app,-,-)
  Datos=read(app.bt,160);
  i=1;j=1;
  y=zeros(1,40); % y es el vector de distancia
  s=zeros(1,40); % s es el vector de estado
  while i<=80
    y(j)=256*Datos(i)+Datos(i+1);
    i=i+2; j=j+1;
  end
  i=81; j=1;
  while (i>=81 && i<=160)
    s(j)=256*Datos(i)+Datos(i+1);
    if (s(j) == 5)
      s(j)=1000;
    else
      s(j)=0;
    end
    i=i+2; j=j+1;
  end
  %disp(y);
  Y = reshape(y,[8,5]); % Y es la matriz de distancia 5 x 8
  Y=Y'; % Matriz traspuesta
  Y = rot90(Y,2);

  Y(Y > 800) = 800;

  S = reshape(s,[8,5]); % S es la matriz de estado 8 x 8
  S=S'; % Matriz traspuesta
  S = rot90(S,2);
  %disp(S);
  Yimg = mat2gray(Y,[0 800]); % Transf. Niveles de grises. 0 negro y 800 blanco
  Sing = mat2gray(S,[0 1000]); % Transf. Niveles de grises. 0 negro y 1000 blanco
  imshow(Yimg,'Parent',app.pltOriginal);
  imshow(Simg,'Parent',app.pltEstado);
```

Figure 17. Código de las reconstrucciones espaciales de las matrices en MATLAB.

Con la matriz ya lista, el filtrado estadístico y la decisión de rumbo en MATLAB entran en juego ejecutándose de forma secuencial. Una vez reconstruida la matriz de 5x8, el código en MATLAB (VL53L5CX_MatlabBT_tercera_prueba) evalúa cada una de las 8 columnas. Para cada columna de 5 celdas, calcula la mediana y la Desviación Absoluta de la Mediana (MAD). Cualquier distancia que se desvíe de la mediana más allá de un umbral ($k=3$ veces la MAD) se considera un error del láser y se descarta (se pone a cero), promediando solo las lecturas válidas. Una vez limpiadas las lecturas malas, MATLAB revisa la zona frontal (columnas centrales 4 y 5): si la distancia cae por debajo de 500 mm, frena el rover a cero; si está libre más allá de 700 mm, le da vía libre (velocidad 70); y en el rango intermedio, interpola la velocidad para frenar suavemente. Si la caja está muy cerca (<600 mm), MATLAB activa el modo de evasión (avoiding = true), busca la columna más despejada y le manda el índice de la columna por Bluetooth al ESP32 para que gire.

En el ESP32, el código del bucle principal (VL53L5CX_tercera_prueba_código_final) espera a recibir ese índice de columna enviado por MATLAB. Al recibirlo, consulta una tabla predefinida (angles_rel) que traduce esa columna a grados reales del campo de visión (por ejemplo, la columna 1 son -24° y la 8 son $+24^\circ$). El ESP32 le suma ese ángulo relativo al rumbo actual y fija una nueva referencia absoluta para el PID (ref_yaw). A partir de ahí, el algoritmo calcula el error angular limitándolo a $\pm 180^\circ$ (wrap180), actualiza el control proporcional e integral limitando su memoria (INTEGRAL_MAX) y suma el esfuerzo resultante a la velocidad base dictada por MATLAB para corregir el rumbo.

Aquí abajo adjunto las capturas de código de estos procesos: a la izquierda, el filtrado estadístico MAD en MATLAB; luego la lógica de decisión de rumbo y velocidad; y a la

derecha, la tabla de ángulos y luego por el último, debajo de todo, el bucle del control PID en Arduino.

```

k = 3;

distMean = zeros(1,8);

for c = 1:8
    x = Y(:,c);

    med = median(x);
    madv = median(abs(x - med));

    x_f = zeros(1,5);
    if madv == 0
        x_f = x;
    else
        for r = 1:5
            variant = abs(x(r) - med);
            if variant <= k*madv
                x_f(r) = x(r);
            else
                x_f(r) = 0;
            end
        end
    end

    valid = x_f(x_f ~= 0);
    if isempty(valid)
        distMean(c) = 0;
    else
        distMean(c) = mean(valid);
    end
end

[~, idx] = max(distMean);

```

Figure 22. Código del filtrado estadístico.

```

% ----- VELOCIDAD BASE -----
% zona frontal
d4 = distMean(4);
d5 = distMean(5);

if d4 > 0 && d5 > 0
    d_front = min(d4, d5);
elseif d4 > 0
    d_front = d4;
elseif d5 > 0
    d_front = d5;
else
    d_front = 0;
end

if d_front <= 500
    speed = 0;
elseif d_front >= 700
    speed = 70;
else
    speed = round((d_front - 500) / 200 * 70);
end

persistent avoiding;
if isempty(avoiding)
    avoiding = false;
end

if d_front <= 600
    avoiding = true; % estoy evitando
    write(app.bt, uint8([4 idx]));
elseif d_front >= 600 && avoiding
    write(app.bt, [6 0]); % vuelve al rumbo inicial
    avoiding = false; % ya no evito
end

% Enviar velocidad a Arduino
write(app.bt, uint8([5 speed]));

if app.running
    write(app.bt, uint8([3 0]));
end
end
end

const float angles_rel[8] = {
    -24.0625, -17.1875, -10.3125, -3.4375,
    3.4375, 10.3125, 17.1875, 24.0625
}; // angulos de yaw respecto a cada columna del sensor

// ----- funciones ángulos -----
float wrap360(float a) {
    while (a >= 360.0) a -= 360.0;
    while (a < 0.0) a += 360.0;
    return a;
}

float wrap180(float a) {
    while (a > 180.0) a -= 360.0;
    while (a < -180.0) a += 360.0;
    return a;
}

```

Figure 23. Código de la extracción de información y decisión de rumbo por parte de Matlab.

```

void loop() {

    // 1) Leer BT (Matlab pide datos o manda ref)
    leerBT();

    if (!started) {
        delay(5);
        return;
    }

    // 2) Leer IMU
    bno.getEvent(&event);
    X = wrap360(event.orientation.x);

    // 3) Error respecto a la referencia absoluta
    error = wrap180(ref_yaw - X);

    // 4) PID
    T = micros();
    delta_T = (float)(T - prev_T) / 1e6;
    prev_T = T;

    integral += error * delta_T;
    if (integral > INTEGRAL_MAX) integral = INTEGRAL_MAX;
    if (integral < -INTEGRAL_MAX) integral = -INTEGRAL_MAX;

    derivator = (error - prev_error) / delta_T;
    prev_error = error;

    u = Kp * error + Ki * integral + Kd * derivator;
    if (u > U_MAX) u = U_MAX;
    if (u < -U_MAX) u = -U_MAX;

    // 5) Motores
    float leftSpeed = BASE_SPEED + u;
    float rightSpeed = BASE_SPEED - u;

    if (leftSpeed < 0) leftSpeed = 0;
    if (leftSpeed > 100) leftSpeed = 100;
    if (rightSpeed < 0) rightSpeed = 0;
    if (rightSpeed > 100) rightSpeed = 100;

    digitalWrite(IN1, LOW);
}

```

Figure 26. Código del control PID en Arduino.

En definitiva, lo que me llevo de este proyecto es haber aprendido a construir un sistema embebido realmente tolerante a fallos: establecer un protocolo maestro-esclavo sincronizado para transferencias de alto volumen, garantizando un vaciado continuo de las colas de recepción (buffers) sin atascos ni retrasos en la transmisión; limpiar con estadística todo ese ruido falso propio de los sensores físicos; y, sobre todo, entender una regla fundamental de arquitectura: siempre que el escenario lo permita (en un cohete, por la necesidad de ser 100% autónomo y por los retardos inadmisibles que causaría mandar datos a tierra, esto se resuelve con hardware embarcado dedicado), delegar los cálculos pesados a un procesador externo o PC. De este modo, evitamos sobrecargar la CPU del microcontrolador principal, garantizando que la tasa de refresco del algoritmo de control no sufra caídas ni retrasos y el vehículo responda de inmediato sin retardos ni bloqueos.